



Computer
Science

CSC 480/580: Principles of Machine Learning

Nonlinear Models

Jason Pacheco

Outline

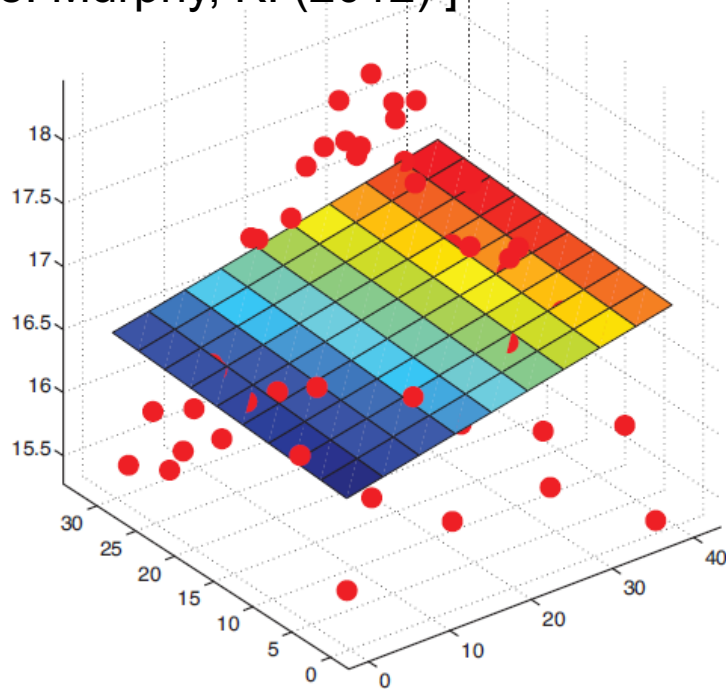
- Basis Functions
- Case study: Support Vector Machine with basis functions
- Kernels

Outline

- **Basis Functions**
- Case study: Support Vector Machine with basis functions
- Kernels

Linear Models

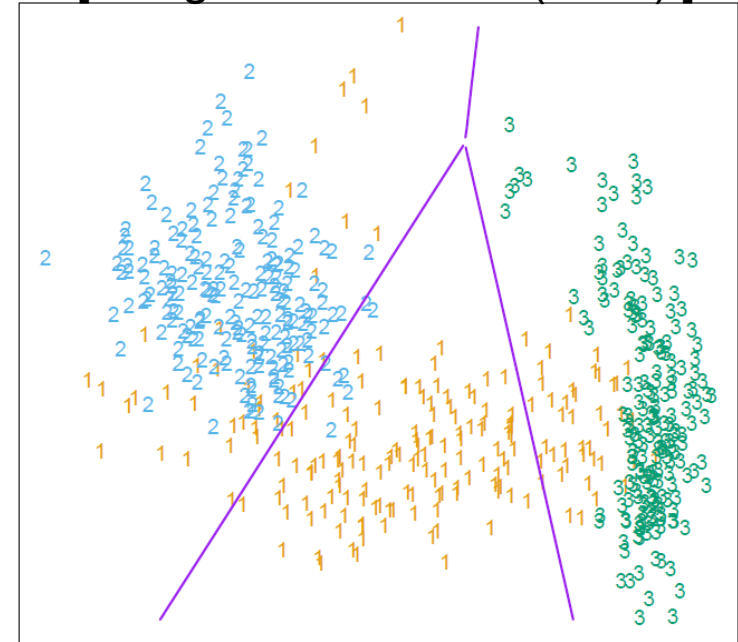
[Image: Murphy, K. (2012)]



Linear Regression Fit a *linear function* to the data,

$$y = w^T x + b$$

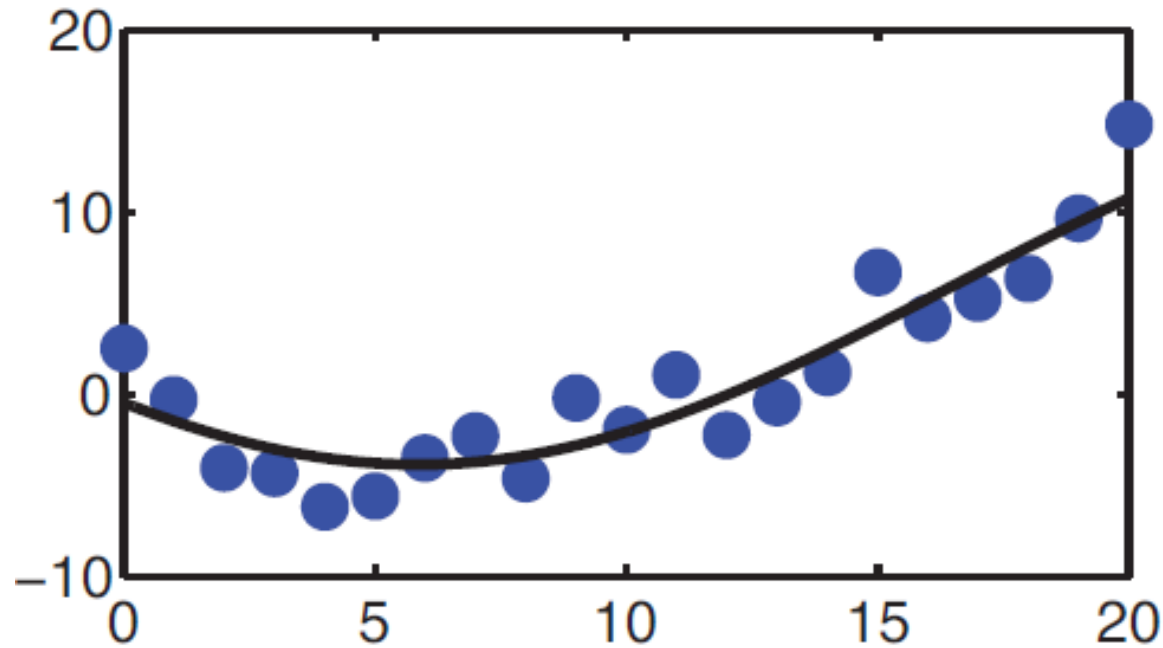
[Image: Hastie et al. (2001)]



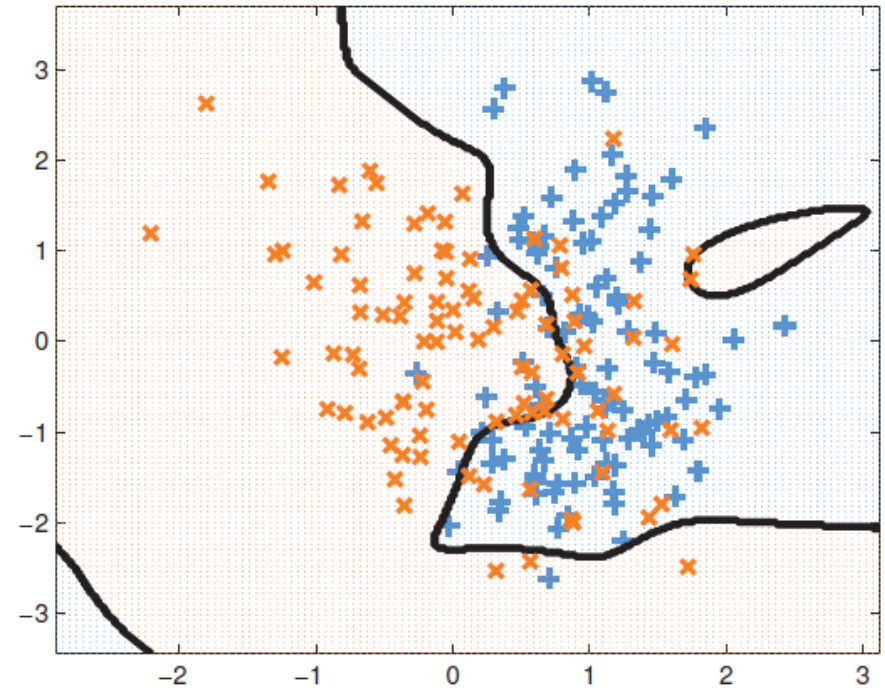
Logistic Regression Learn a decision boundary that is *linear in the data*,

$$P(y = 1 \mid w, x) = \sigma(w^T x)$$

Nonlinear Data



What if our data are *not* well-described by a linear function?

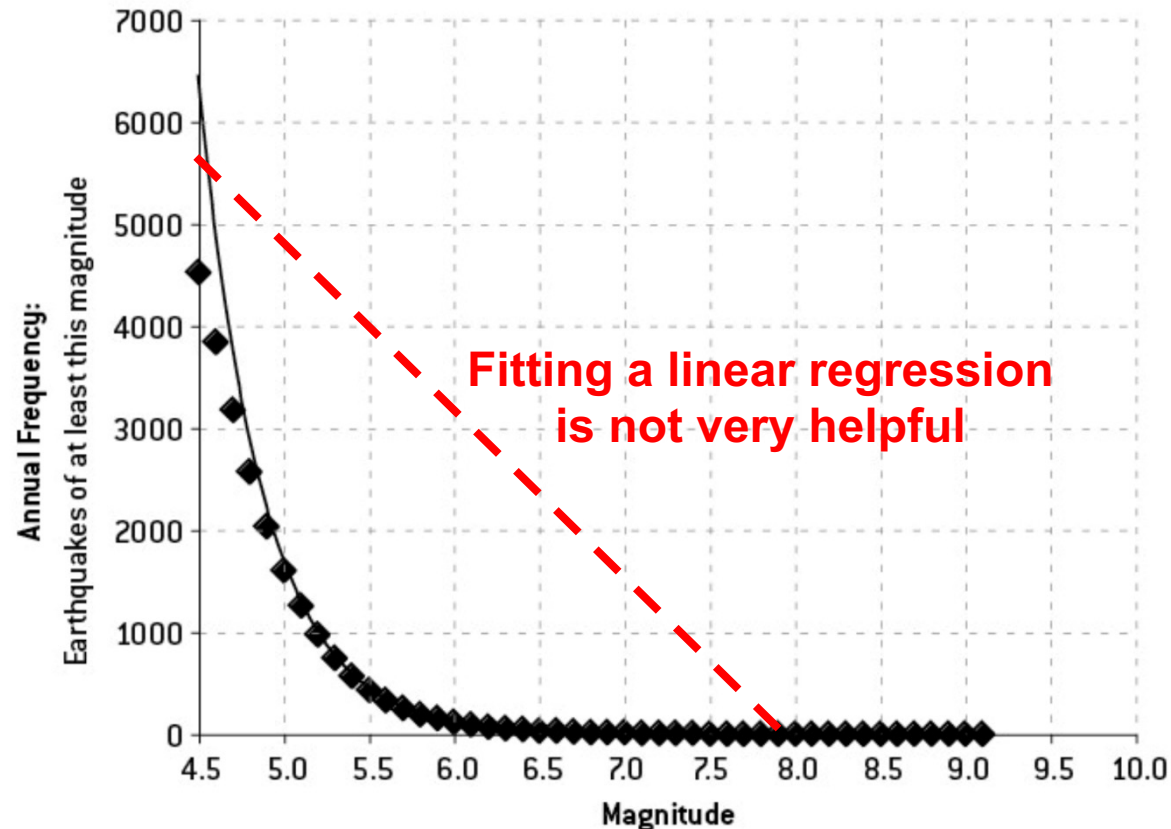


What if classes are *not linearly-separable*?

Example: Earthquake Prediction

Suppose that we want to predict the number of earthquakes that occur of a certain magnitude. Our data are given by,

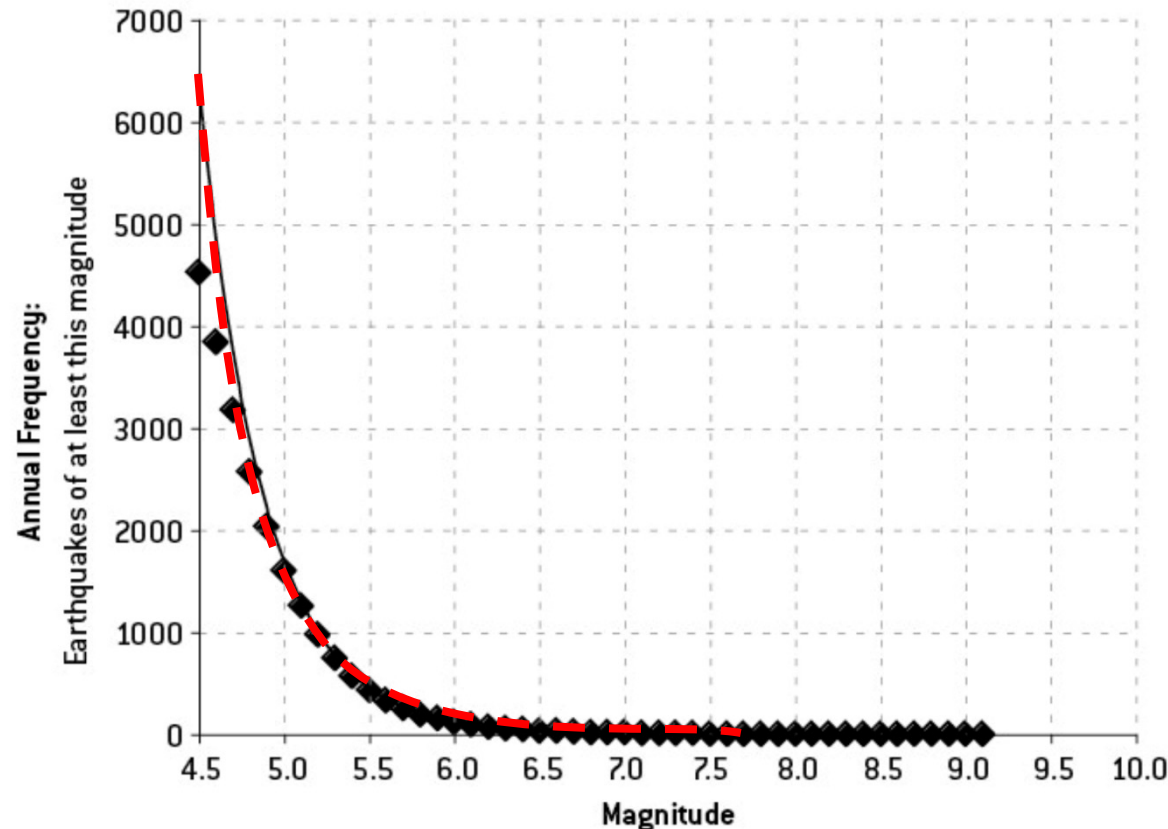
FIGURE 5-3A: WORLDWIDE EARTHQUAKE FREQUENCIES, JANUARY 1964–MARCH 2012



Example: Earthquake Prediction

Suppose that we want to predict the number of earthquakes that occur of a certain magnitude. Our data are given by,

FIGURE 5-3A: WORLDWIDE EARTHQUAKE FREQUENCIES, JANUARY 1964–MARCH 2012



Idea Instead of fitting ordinary linear regression,

$$y = w^T x$$

Fit an alternative model

$$\ln(y) = w^T x$$

Basis Functions

- A **basis function** can be any function of the input features X
- Define a set of m basis functions $\phi_1(x), \dots, \phi_m(x)$
- Fit a linear regression model in terms of basis functions,

$$y = \sum_{i=1}^m w_i \phi_i(x) = w^T \phi(x)$$

- Regression model is *linear in the basis transformations*
- Model is *nonlinear in the data X*

Common “All-Purpose” Basis Functions



- Linear basis functions recover the original linear model,

$$\phi_m(x) = x_m$$

Returns m^{th} dimension of X

- Quadratic $\phi_m(x) = x_j^2$ or $\phi_m(x) = x_j x_k$ capture 2nd order interactions
- An order p polynomial $\phi \rightarrow x_d, x_d^2, \dots, x_d^p$ captures higher-order nonlinearities (but requires $O(d^p)$ parameters)
- Nonlinear transformation of single inputs,

$$\phi \rightarrow (\log(x_j), \sqrt{x_j}, \dots)$$

- An indicator function specifies a region of the input,

$$\phi_m(x) = I(L_m \leq x_k < U_m)$$

sklearn.preprocessing.PolynomialFeatures

degree : int or tuple (min_degree, max_degree), default=2

If a single int is given, it specifies the maximal degree of the polynomial features. If a tuple (min_degree, max_degree) is passed, then min_degree is the minimum and max_degree is the maximum polynomial degree of the generated features. Note that min_degree=0 and min_degree=1 are equivalent as outputting the degree zero term is determined by include_bias.

interaction_only : bool, default=False

If True, only interaction features are produced: features that are products of at most degree distinct input features, i.e. terms with power of 2 or higher of the same input feature are excluded:

- included: $x[0]$, $x[1]$, $x[0] * x[1]$, etc.
- excluded: $x[0] ** 2$, $x[0] ** 2 * x[1]$, etc.

include_bias : bool, default=True

If True (default), then include a bias column, the feature in which all polynomial powers are zero (i.e. a column of ones - acts as an intercept term in a linear model).

order : {'C', 'F'}, default='C'

Order of output array in the dense case. 'F' order is faster to compute, but may slow down subsequent estimators.

Example: Polynomial Basis Functions

Create three two-dimensional data points [0,1], [2,3], [4,5]:

```
>>> X = np.arange(6).reshape(3, 2)
>>> X
array([[0, 1],
       [2, 3],
       [4, 5]])
```

Compute quadratic features $(1, x_1, x_2, x_1^2, x_1x_2, x_2^2)$,

```
>>> poly = PolynomialFeatures(degree=2)
>>> poly.fit_transform(X)
array([[ 1.,  0.,  1.,  0.,  0.,  1.],
       [ 1.,  2.,  3.,  4.,  6.,  9.],
       [ 1.,  4.,  5., 16., 20., 25.]])
```

These are now our new data and ready to fit a model...

Example: Polynomial Regression

Create a 3rd order polynomial (cubic) regression problem,

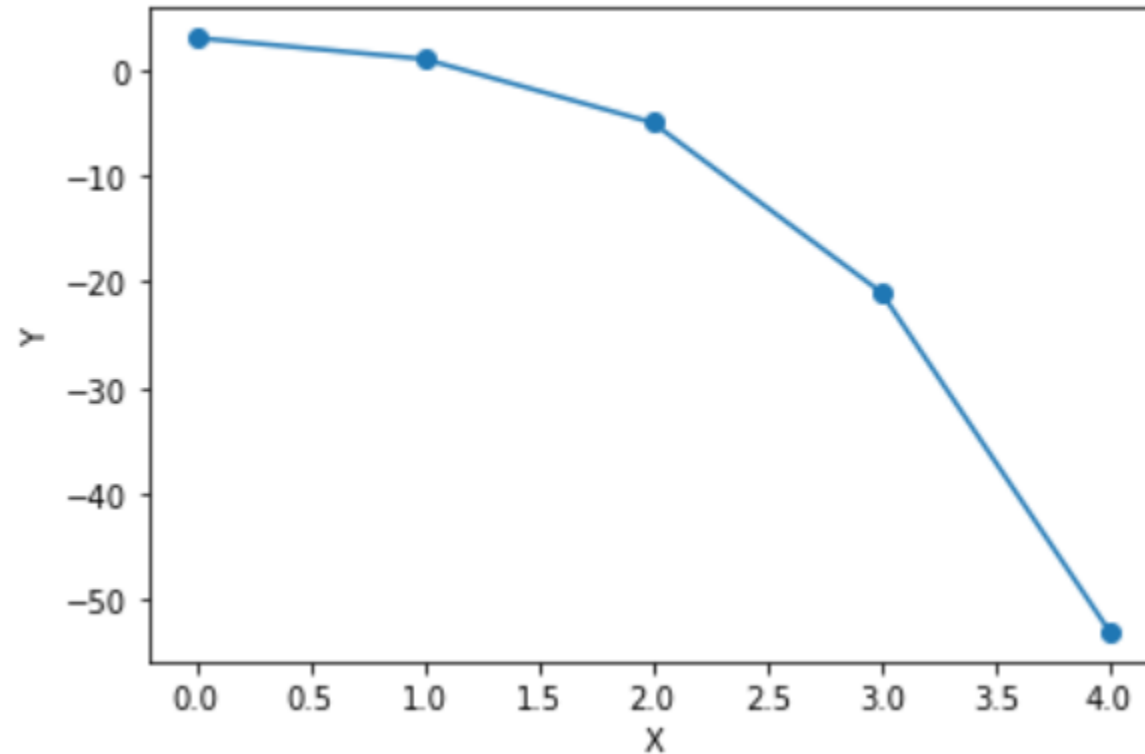
```
from sklearn.preprocessing import PolynomialFeatures
x = np.arange(5)
y = 3 - 2 * x + x ** 2 - x ** 3
y
array([ 3,  1, -5, -21, -53])
```

Create cubic features $(1, x, x^2, x^3)$,

```
from sklearn.linear_model import LinearRegression
poly = PolynomialFeatures(degree=3)
x_new = poly.fit_transform(x[:,np.newaxis])
x_new
array([[ 1.,  0.,  0.,  0.],
       [ 1.,  1.,  1.,  1.],
       [ 1.,  2.,  4.,  8.],
       [ 1.,  3.,  9., 27.],
       [ 1.,  4., 16., 64.]])
```

Example: Polynomial Regression

```
model = LinearRegression(fit_intercept=False).fit(x_new, y)
ypred = model.predict(x_new)
plt.scatter(x, y)
plt.plot(x, ypred, '-')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```



Linear Regression

Recall that the ordinary least squares solution is given by,

$$\mathbf{X} = \begin{pmatrix} 1 & x_{11} & \dots & x_{1D} \\ 1 & x_{21} & \dots & x_{2D} \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_{N1} & \dots & x_{ND} \end{pmatrix} \quad \mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_N \end{pmatrix} \quad w^{\text{OLS}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Design Matrix
(each training input on a column)

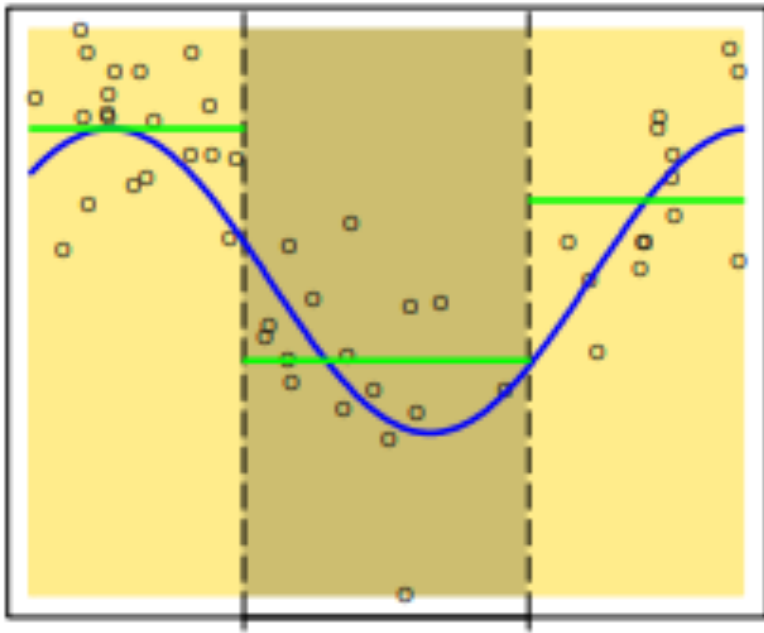
Vector of
Training labels

Can similarly solve in terms of basis functions,

$$\mathbf{\Phi} = \begin{pmatrix} 1 & \phi_1(x_1) & \dots & \phi_M(x_1) \\ 1 & \phi_1(x_2) & \dots & \phi_M(x_2) \\ \vdots & \vdots & \vdots & \vdots \\ 1 & \phi_1(x_N) & \dots & \phi_M(x_N) \end{pmatrix} \quad w^{\text{OLS}} = (\mathbf{\Phi}^T \mathbf{\Phi})^{-1} \mathbf{\Phi}^T \mathbf{y}$$

Example: Piecewise Constant Regression

[Source: Hastie et al. (2001)]



Decompose the input space into 3 regions with indicator basis functions,

$$\phi_1(x) = I(x < \xi_1)$$

$$\phi_2(x) = I(\xi_1 \leq x < \xi_2)$$

$$\phi_3(x) = I(\xi_2 \leq x)$$

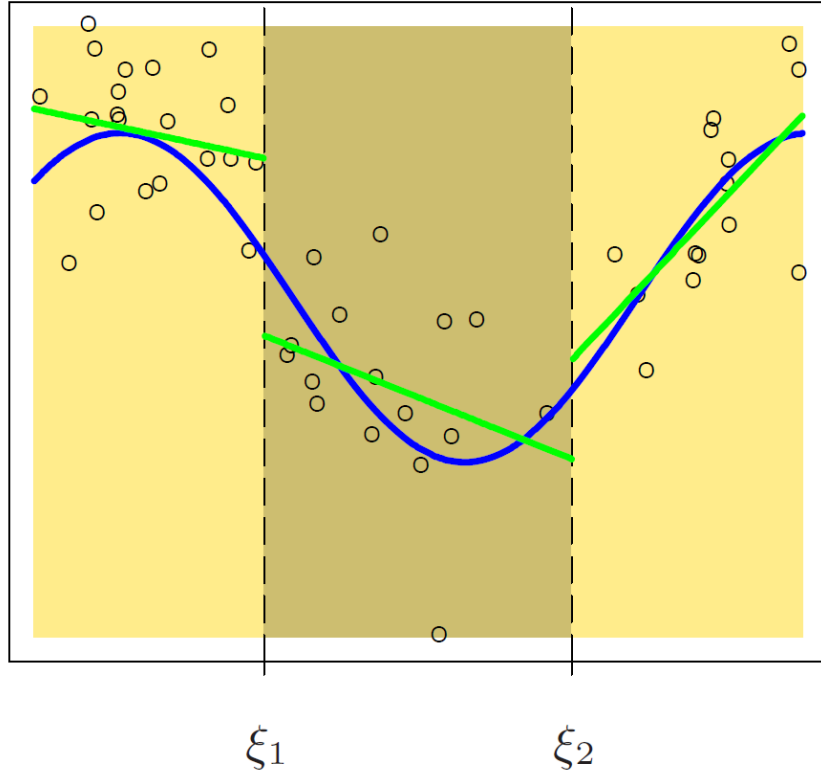
Fit linear regression model,

$$y = w_1\phi_1(x) + w_2\phi_2(x) + w_3\phi_3(x)$$

Effectively fits 3 constant functions to data in each region

Example: Piecewise Linear Regression

[Source: Hastie et al. (2001)]



**Regression lines are discontinuous
at boundary points**

Decompose the input space into 3 regions with basis functions,

$$\phi_1(x) = I(x < \xi_1) \quad \phi_4(x) = xI(x < \xi_1)$$

$$\phi_2(x) = I(\xi_1 \leq x < \xi_2) \quad \phi_5(x) = xI(\xi_1 \leq x < \xi_2)$$

$$\phi_3(x) = I(\xi_2 \leq x) \quad \phi_6(x) = xI(\xi_2 \leq x)$$

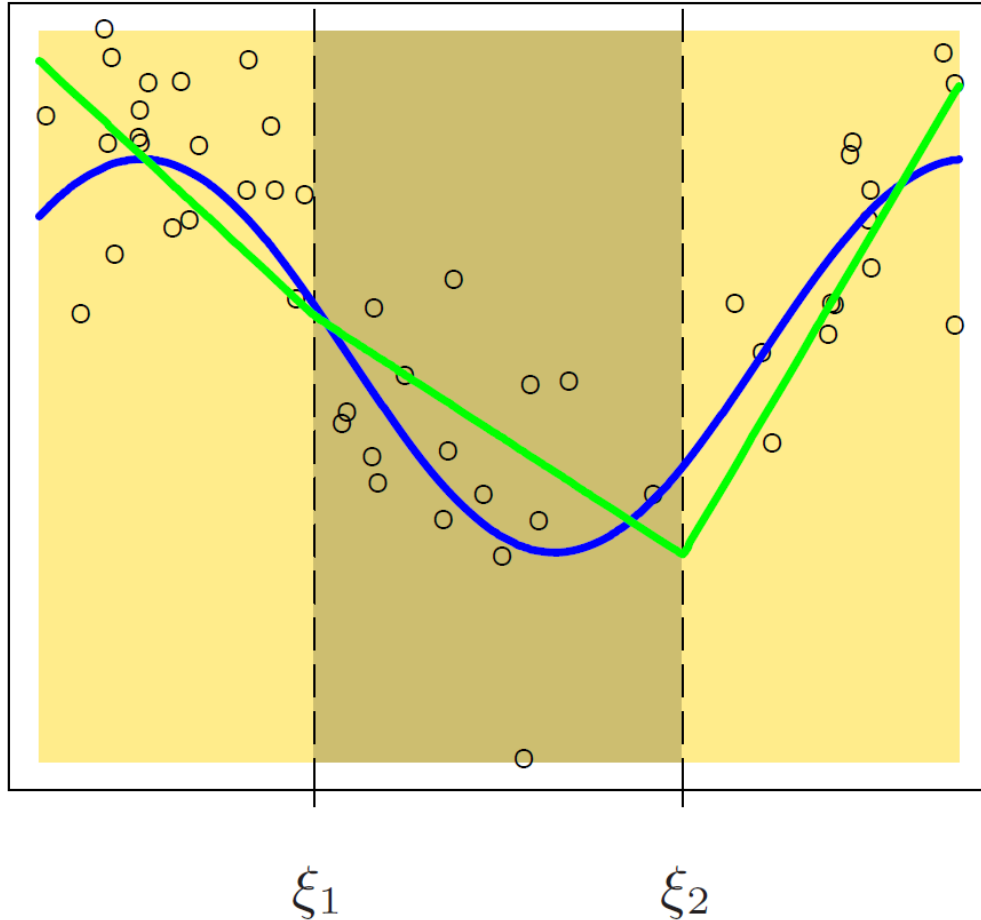
Fit linear regression model,

$$y = \sum_{i=1}^6 w_i \phi_i(x)$$

Effectively fits 3 linear regressions independently to data in each region

Example: Piecewise Linear Regression

[Source: Hastie et al. (2001)]



Enforce constraint that lines agree at boundary points,

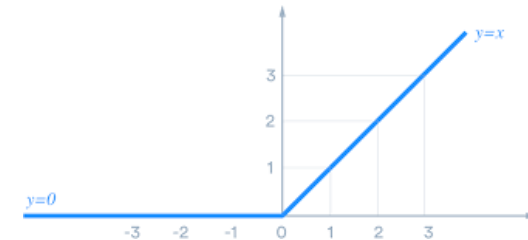
$$\phi_1(x) = 1$$

$$\phi_2(x) = x$$

$$\phi_3(x) = (x - \xi_1)_+$$

$$\phi_4(x) = (x - \xi_2)_+$$

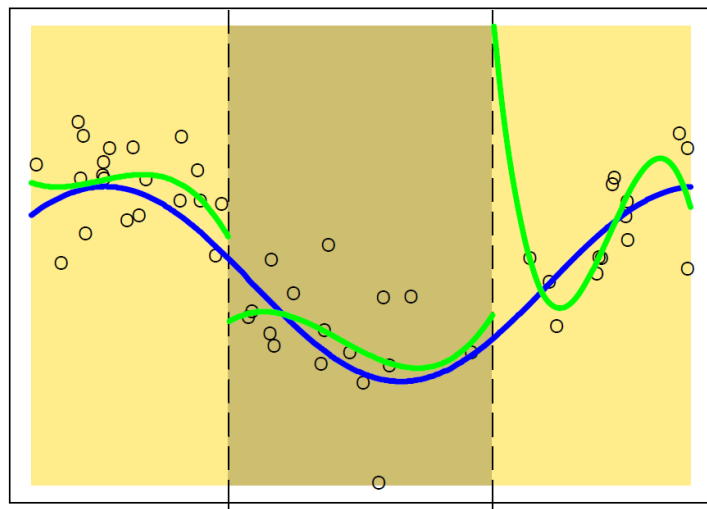
Where $(z)_+ := \max(z, 0)$



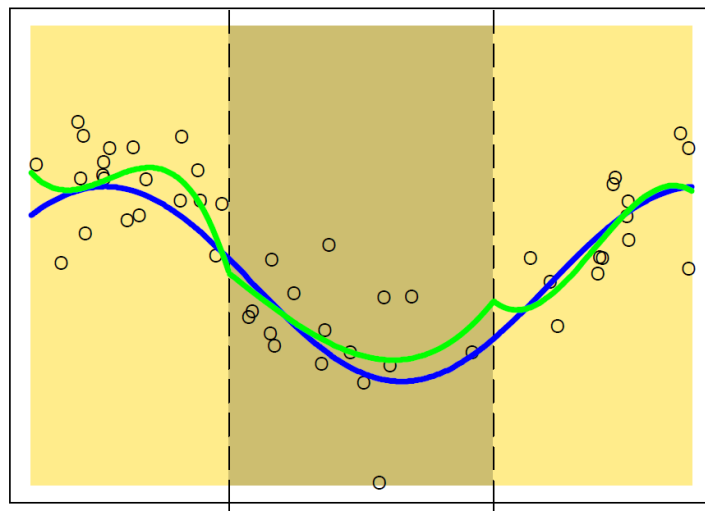
Takeaway: basis functions can encode the inductive biases of the learner
An improvement, but generally prefer *smoother* functions...

[Source: Hastie et al. (2001)]

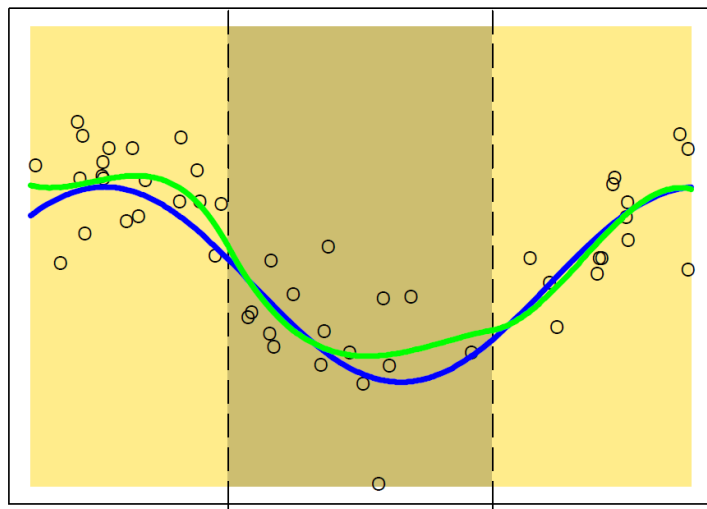
Discontinuous



Continuous



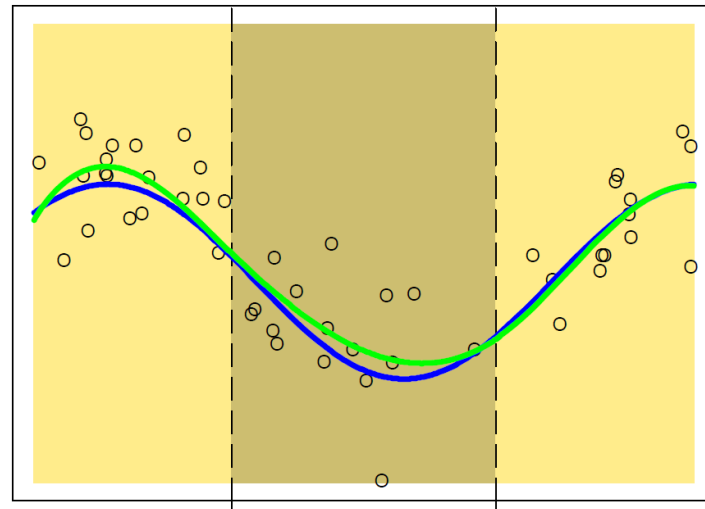
Continuous First Derivative



ξ_1

ξ_2

Continuous Second Derivative



ξ_1

ξ_2

Replace linear basis functions with polynomial,

$$\phi_1(x) = 1 \quad \phi_2(x) = x$$

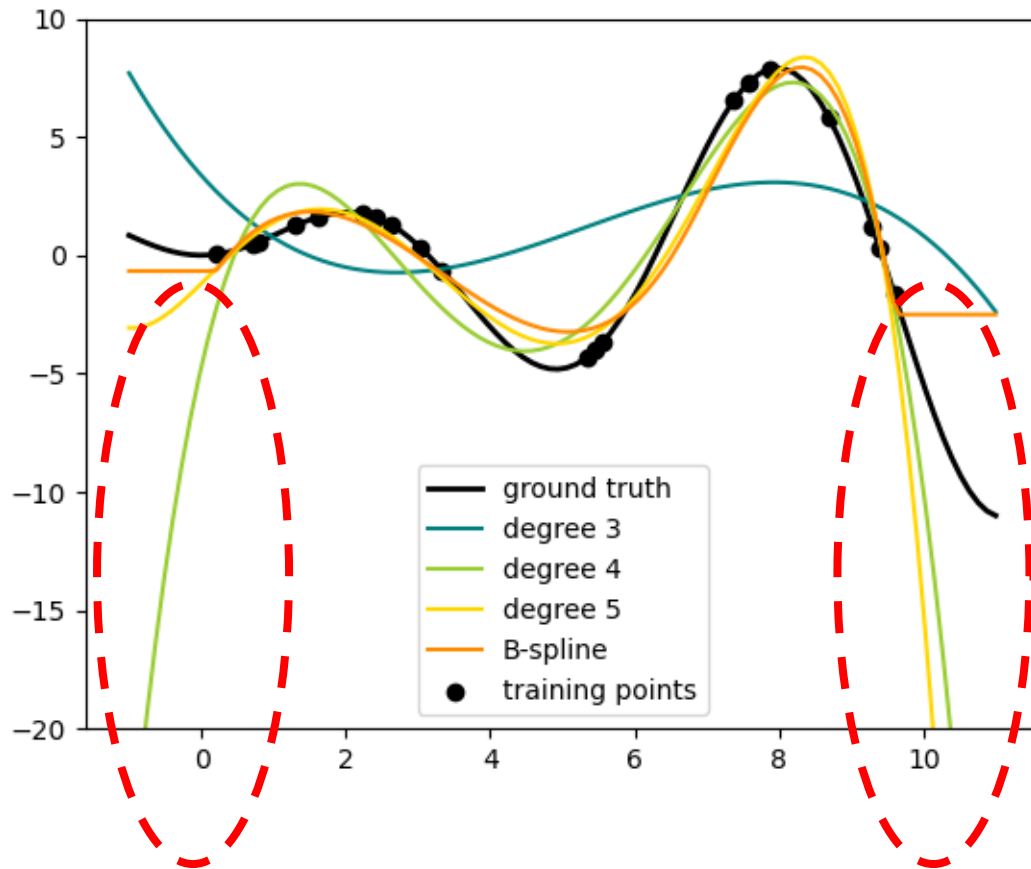
$$\phi_3(x) = x^2 \quad \phi_4(x) = x^3$$

$$\phi_5(x) = (x - \xi_1)_+^3$$

$$\phi_6(x) = (x - \xi_2)_+^3$$

Additional constraints ensure smooth 1st and 2nd derivatives at boundaries

Polynomial Splines



These piecewise regression functions are called *splines*

Supported in Scikit-Learn
`preprocessing.SplineTransformer`

Caution Polynomial basis functions often yield poor out-of-sample predictions with higher order producing more extreme predictions (Runge's Phenomenon, 1901)

Data Preprocessing

- Generally the first step in data science involves *preprocessing* or transforming data in some way
 - Filling in missing values (imputation)
 - Centering / normalizing data
 - Etc.
- We then fit our models to this preprocessed data
- One way to view preprocessing is simply as computing some basis function $\phi(x)$

Basis Functions

Pros

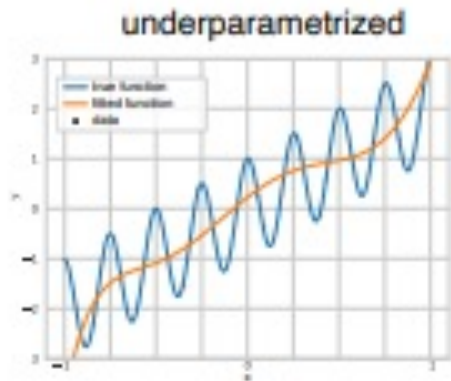
- More flexible modeling that is nonlinear in the original data
- Increases model expressivity

Cons

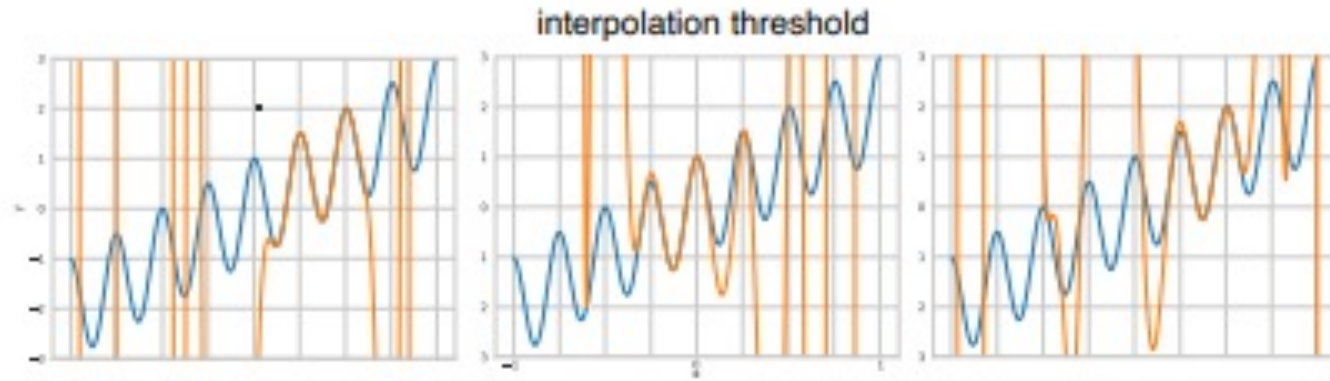
- Typically requires more parameters to be learned
- More sensitive to overfitting training data – needs regularization
- Need to find *good* basis functions (feature engineering)

Overfitting can happen (Schaeffer et al, 2023)

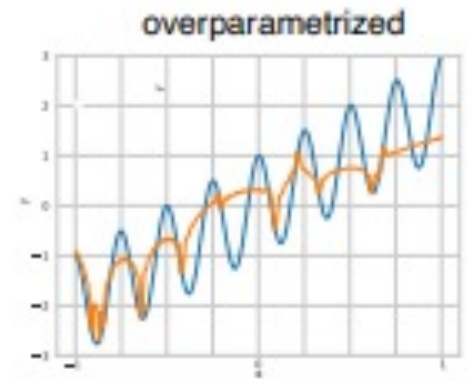
- $y = 2x + \cos(25x)$
- Fit a linear regression model with polynomial feature map $\phi(x) = (1, x, \dots, x^p)$ with 20 training examples



p small



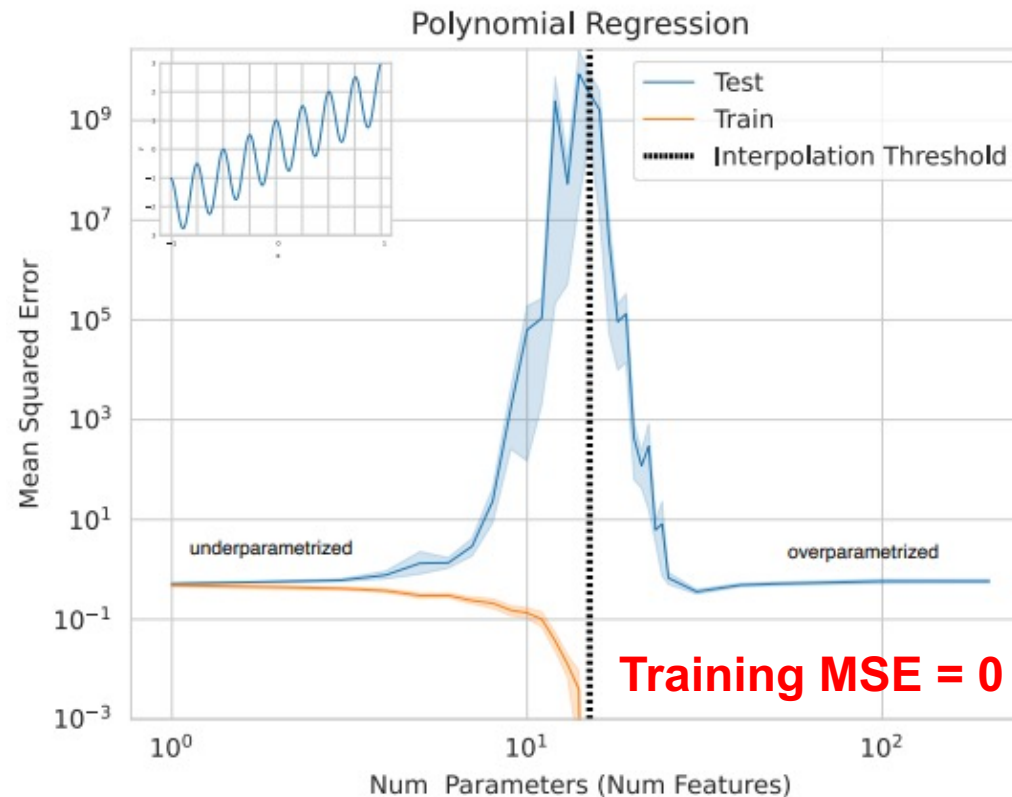
$p \approx 20$ -- overfitting



$p = 200$ – “benign overfitting”

Overfitting can happen (Schaeffer et al, 2023)

- $y = 2x + \cos(25x)$
- Fit a linear regression model with polynomial feature map $\phi(x) = (1, x, \dots, x^p)$ with 20 training examples



Training MSE = 0 whenever $p \geq 20$

<https://arxiv.org/pdf/2303.14151.pdf>

Classical regime

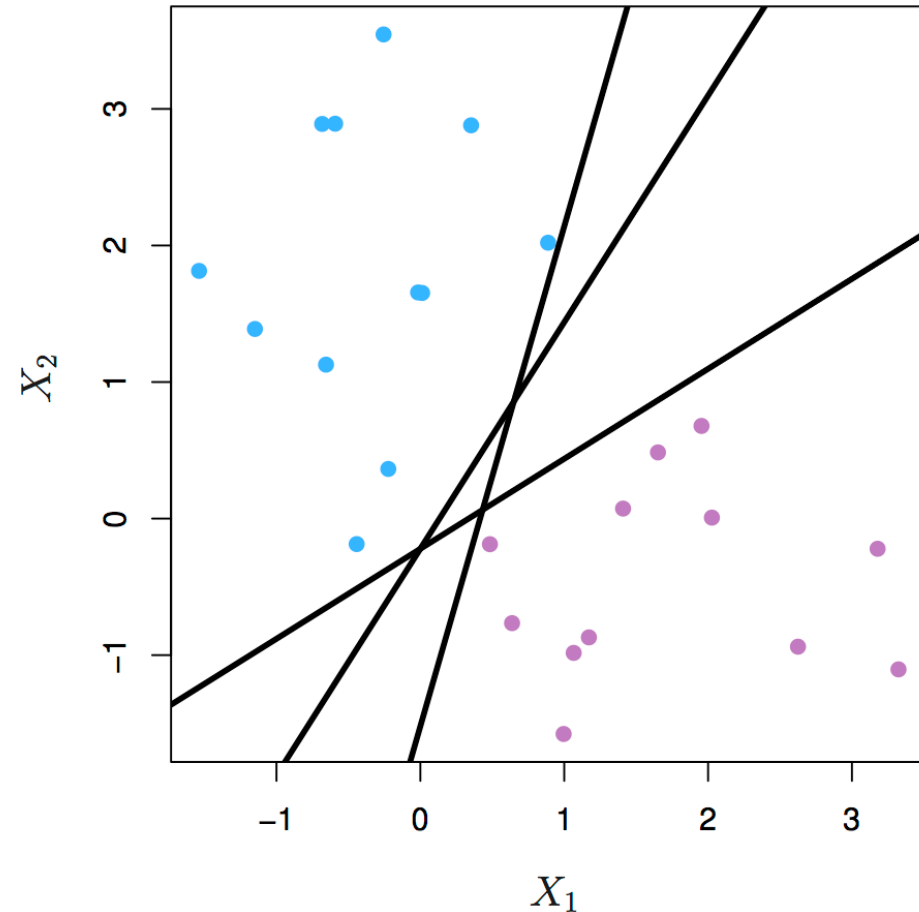
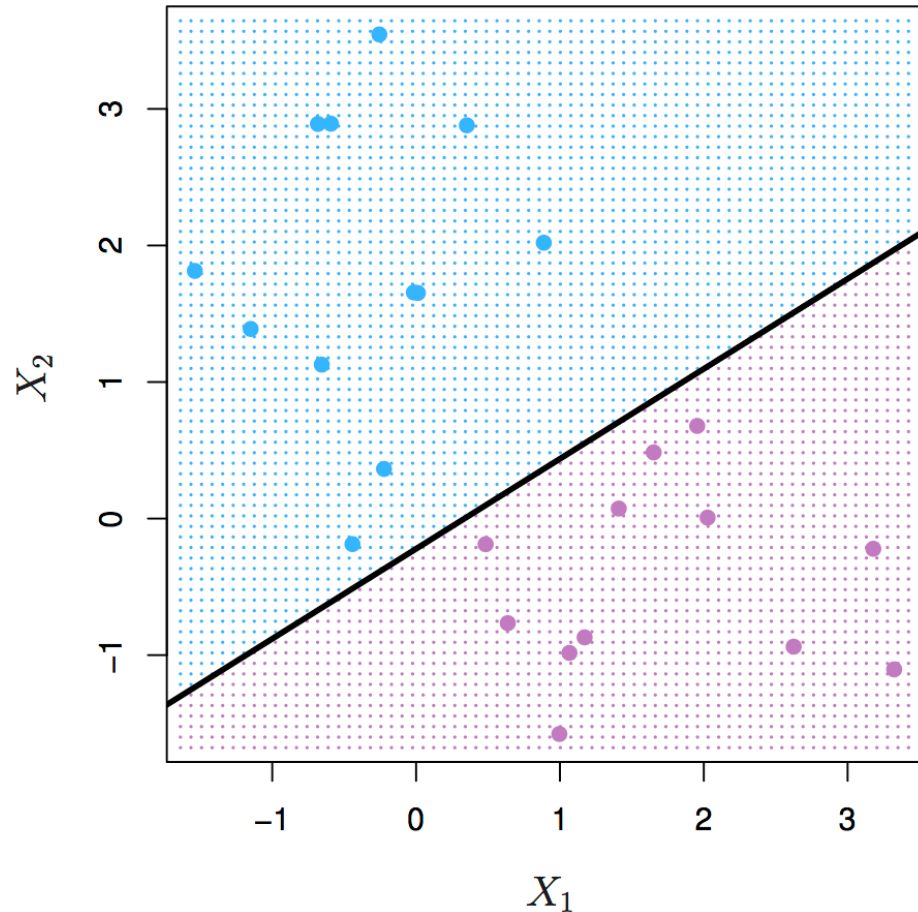
**Modern regime – benign overfitting
coined by (Belkin et al, 2018)**

Outline

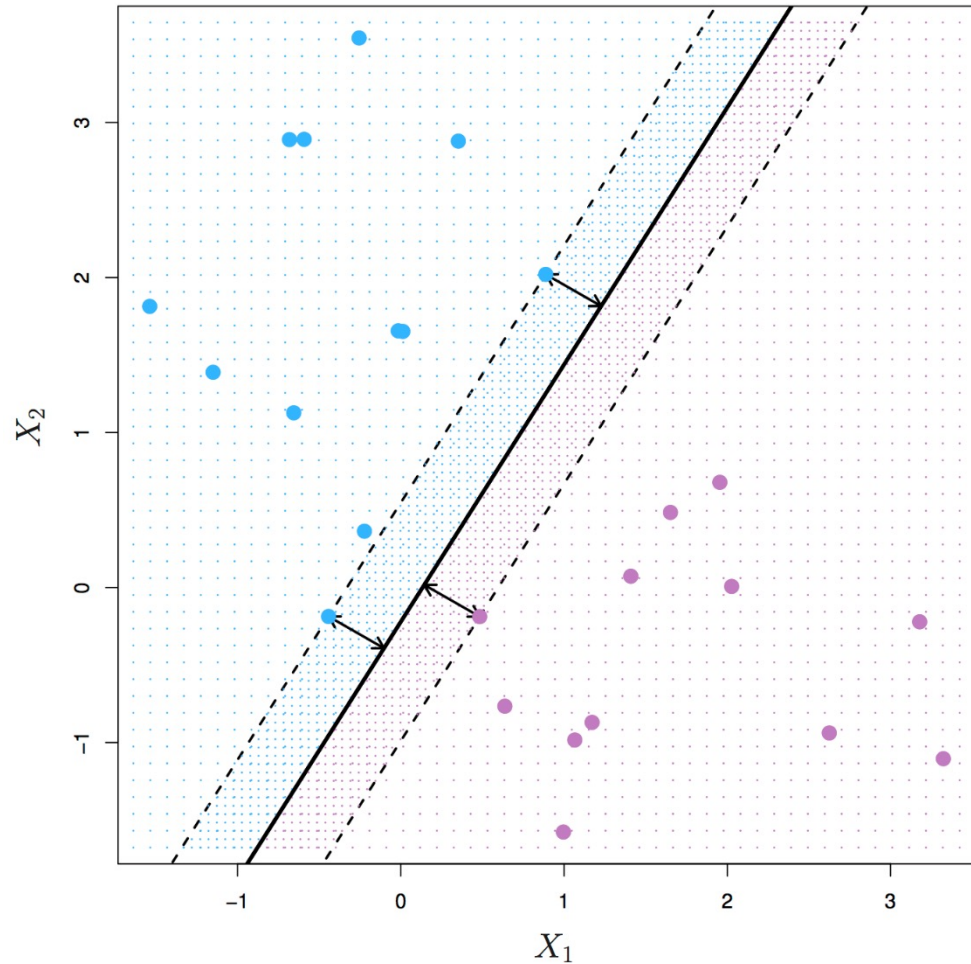
- Basis Functions
- **Case study: Support Vector Machine with basis functions**
- Kernels

Linear Decision Boundary

Any boundary that separates classes is equally good on training data
Are they equally good on unseen test data? **Not necessarily**



Classifier Margin



*The **margin** measures minimum distance between each class and the decision boundary*

Observation Decision boundaries with larger margins are more likely to generalize to unseen data

Idea Learn the classifier with the largest margin that still separates the data...

...we call this a *max-margin classifier*

Computing the Margin

Recall that the linear model is given by

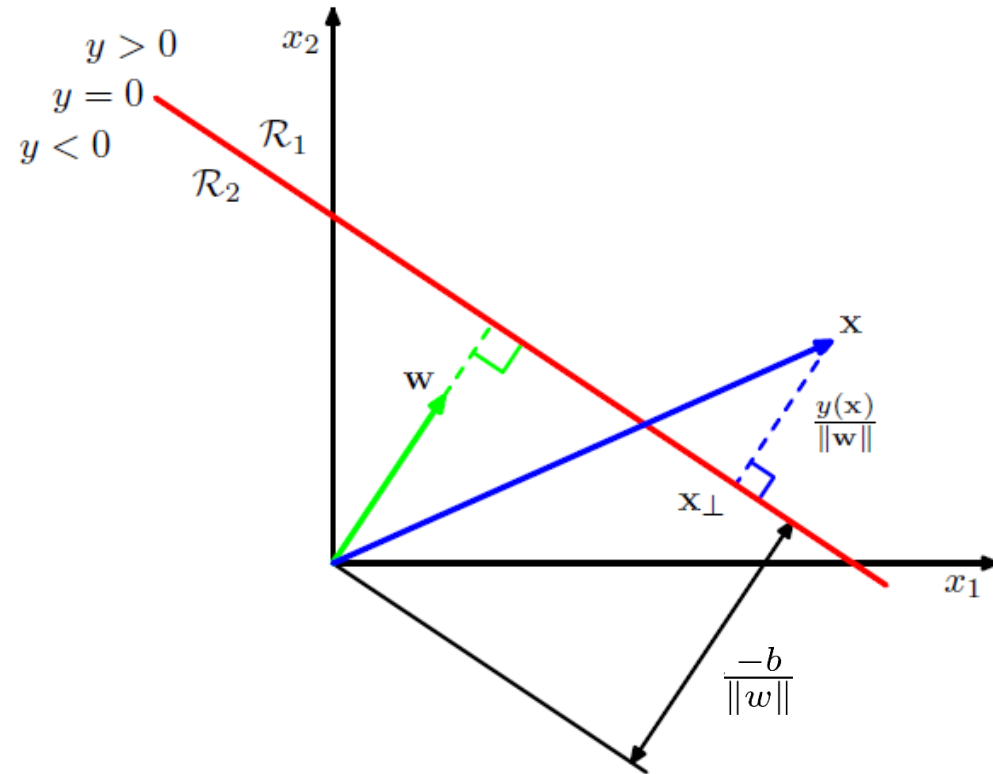
$$y(x) = w^T \phi(x) + b$$

Decision boundary is now at $y(x) = 0$ and **signed** distance of $\phi(x)$ to the boundary is:

$$\frac{y(x)}{\|w\|} = \frac{w^T \phi(x) + b}{\|w\|}$$

Where the norm of the weights is

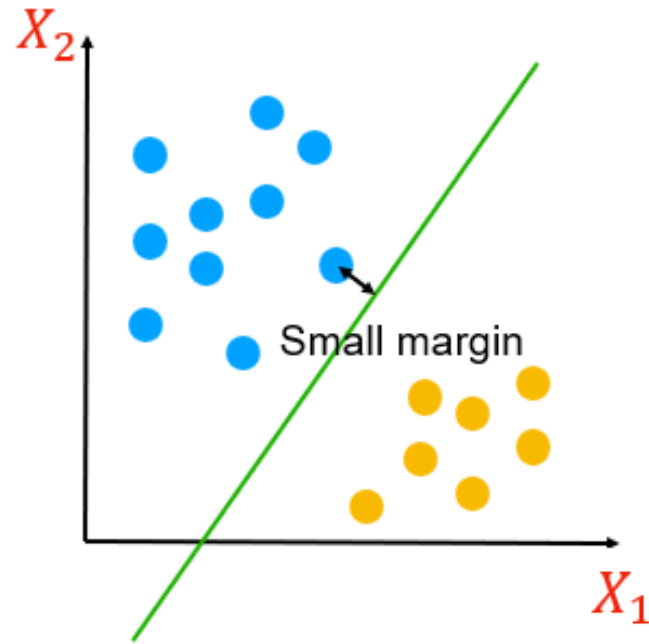
$$\|w\| = \sqrt{w^T w} = \sqrt{\sum_i w_i^2}$$



Recall HW0 -- Also known as the *distance from a point to a plane* equation:

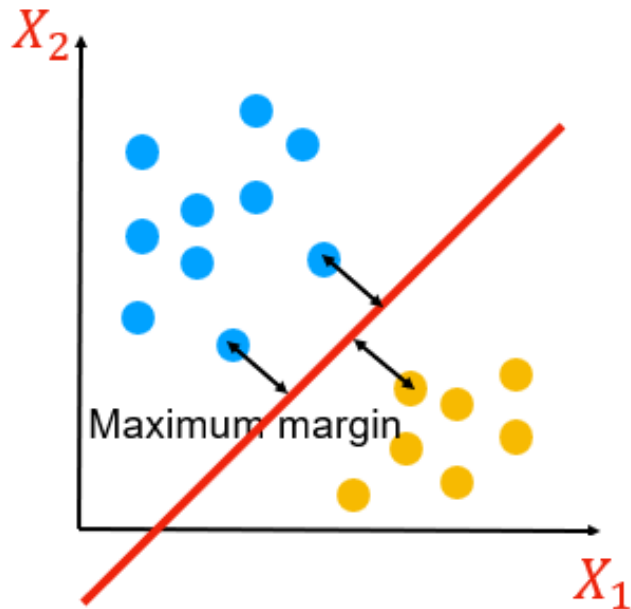
[wiki/Distance from a point to a plane](https://en.wikipedia.org/wiki/Distance_from_a_point_to_a_plane)

Support Vector Machines (SVM)



The **margin** measures minimum distance between each class and the decision boundary

Observation Decision boundaries with larger margins are more likely to generalize to unseen data



Idea Learn the classifier with the largest margin that still separates the data...

...we call this a *max-margin classifier*

Max-Margin Classifier

For training data $\{(x_n, y_n)\}$, we would like to choose (w, b) to ensure two properties:

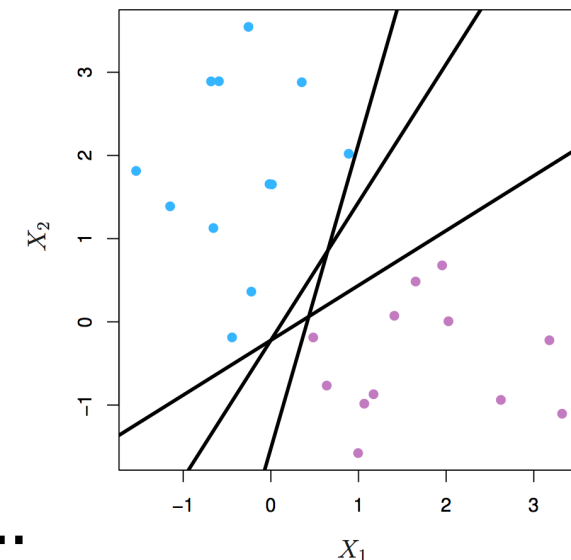
(1) all points are correctly classified:

$$y_n (w^\top \phi(x_n) + b) > 0, n = 1, \dots, N$$

(2) the margins of all points $\frac{|w^\top \phi(x_n) + b|}{\|w\|}$, $n = 1, \dots, N$ are as large as possible

This motivates the following optimization problem (O1):

$$\begin{aligned} & \underset{w, b}{\text{maximize}} \quad \left(\min_n \frac{|w^\top \phi(x_n) + b|}{\|w\|} \right) \\ & \text{subject to: } y_n (w^\top \phi(x_n) + b) > 0, n = 1, \dots \end{aligned}$$



Support Vector Machine (SVM) derivation

The objective is not a convex function.. Let's try to convert it to a convex function..

$$(O1) \quad \underset{w, b}{\text{maximize}} \quad \left(\min_n \frac{|w^\top \phi(x_n) + b|}{\|w\|} \right) \quad \text{Minimum margin}$$

$$\text{subject to: } y_n (w^\top \phi(x_n) + b) > 0, n = 1, \dots, N \quad \text{Perfect classification}$$

The constraint set is OK.. It's a collection of linear inequalities

- Note: for perfect classification, $\frac{|w^\top \phi(x_n) + b|}{\|w\|} = \frac{y_n (w^\top \phi(x_n) + b)}{\|w\|}$ for all n

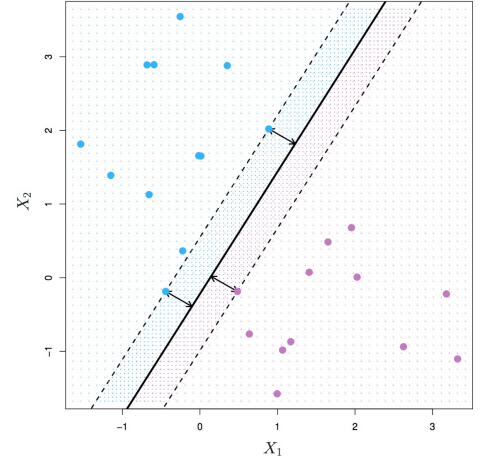
- Therefore, (O1) is equivalent to (O2):

$$\underset{w, b}{\text{maximize}} \quad \left(\min_n \frac{y_n (w^\top \phi(x_n) + b)}{\|w\|} \right)$$
$$\text{subject to: } y_n (w^\top \phi(x_n) + b) > 0, n = 1, \dots, N$$

SVM derivation

- (O2):

$$\begin{aligned} & \underset{w, b}{\text{maximize}} \quad \left(\min_n \frac{y_n (w^\top \phi(x_n) + b)}{\|w\|} \right) \\ & \text{subject to: } y_n (w^\top \phi(x_n) + b) > 0, n = 1, \dots \end{aligned}$$



- Infinitely many solutions – e.g., if (w, b) is optimal, then $(2w, 2b)$ is also optimal
- Let's add the constraint: $\min_n y_n (w^\top \phi(x_n) + b) = 1$

SVM derivation (580 only)

To solve (O2), it suffices to solve (O3):

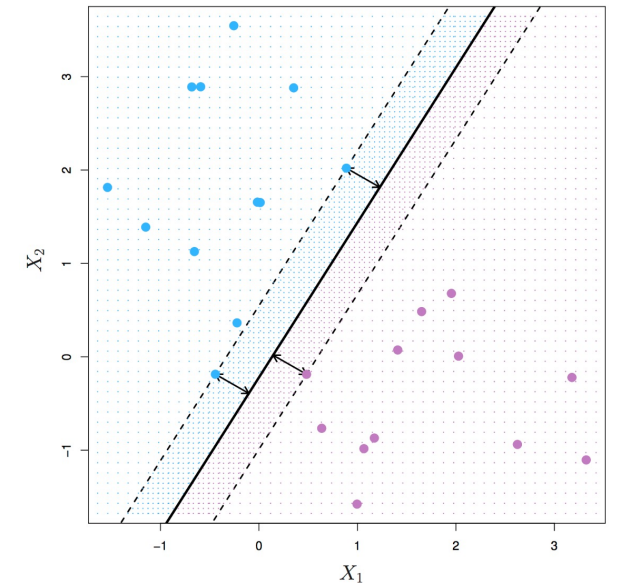
$$\begin{aligned} & \underset{w, b}{\text{maximize}} \quad \left(\min_n \frac{y_n (w^\top \phi(x_n) + b)}{\|w\|} \right) \\ & \text{subject to: } \min_n y_n (w^\top \phi(x_n) + b) = 1 \end{aligned}$$

Observe: this is now $= \frac{1}{\|w\|}$!

Rewrite this as:

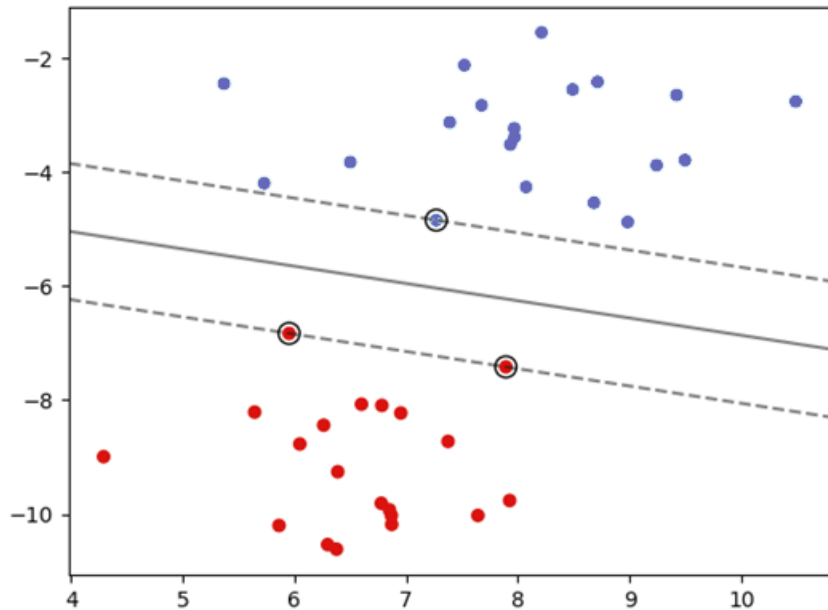
$$\begin{aligned} & \underset{w, b}{\min} \quad \frac{1}{2} \|w\|^2 \\ & \text{s. t. } y_i (w^\top x_i + b) \geq 1, \forall i \end{aligned}$$

This is the SVM Primal Optimization



Support Vector Machine

SVM solves the **convex optimization problem**...



$$\begin{aligned} &\text{minimize } \frac{1}{2} \|w\|^2 \\ &\text{subject to} \end{aligned}$$

This is known as the
primal optimization

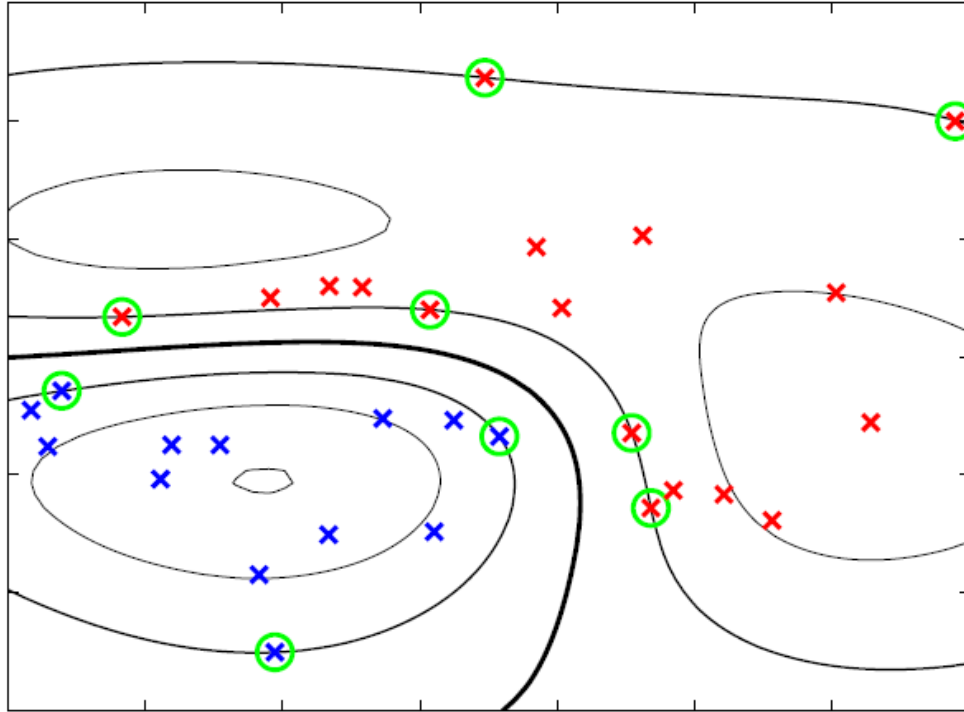
$$y_n(w^T x_n + b) \geq 1 \quad \text{for } n = 1, \dots, N$$

Convex optimization problems can generally be solved efficiently (e.g. CVXPY)

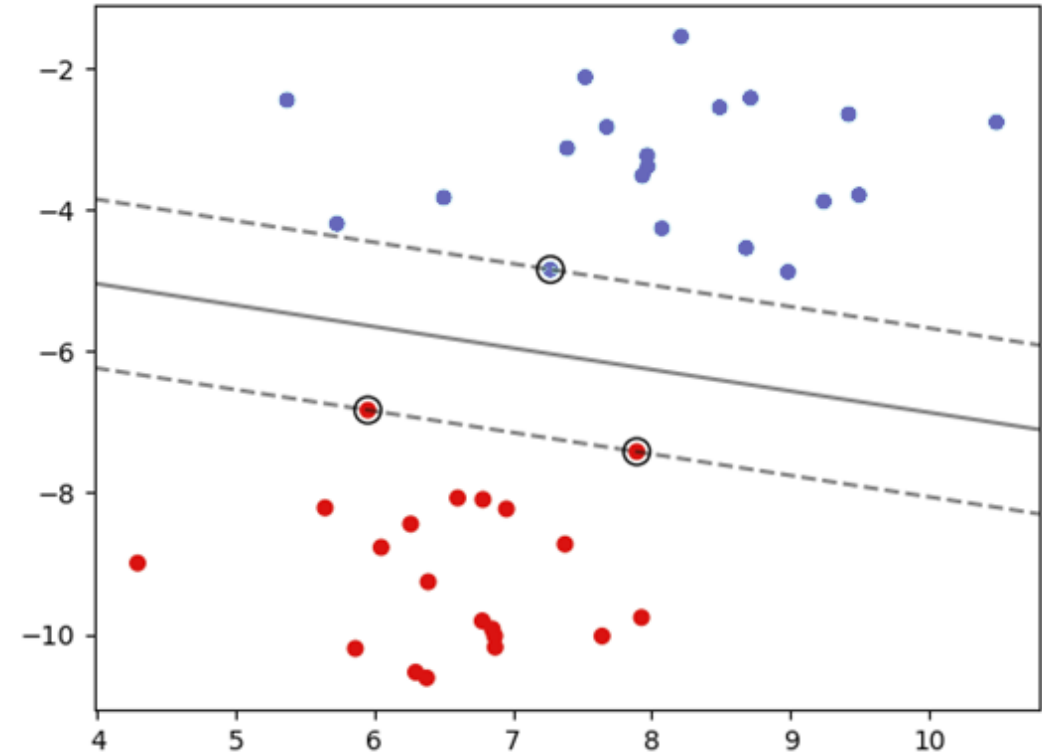
- x are d -dimensional *vectors* $\Rightarrow w$ is d -dimensional vector
- Margins determined by nearest data points called *support vectors*

Nonlinear Max-Margin Classifier

Data Space



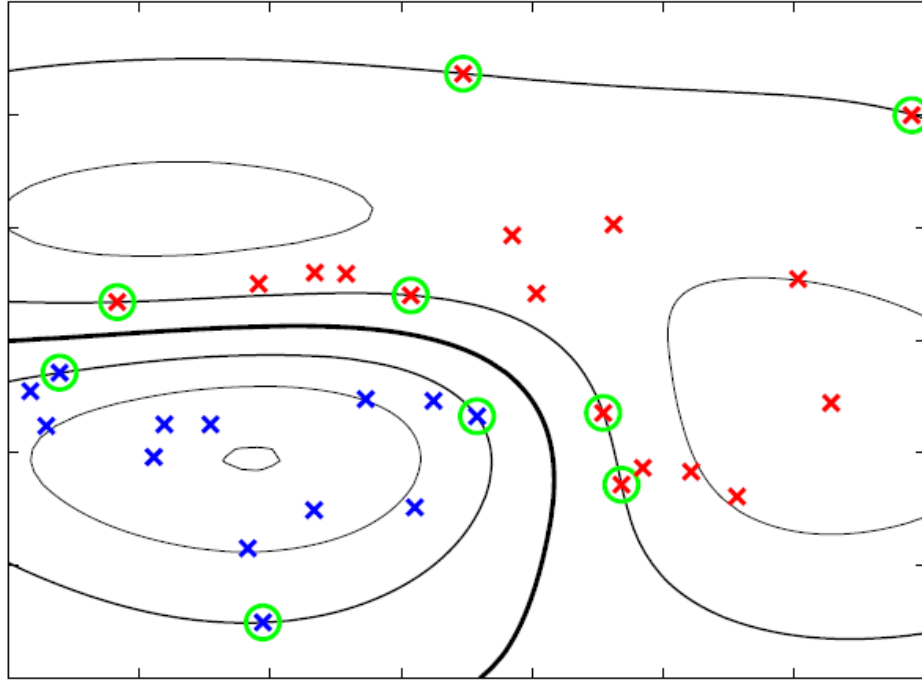
Basis Space



Decision boundary is linear in the transformed data, but nonlinear in the original data space

Nonlinear SVM: SVM with basis function

Again, this is equivalent to:



$$\text{minimize } \frac{1}{2} \|w\|^2$$

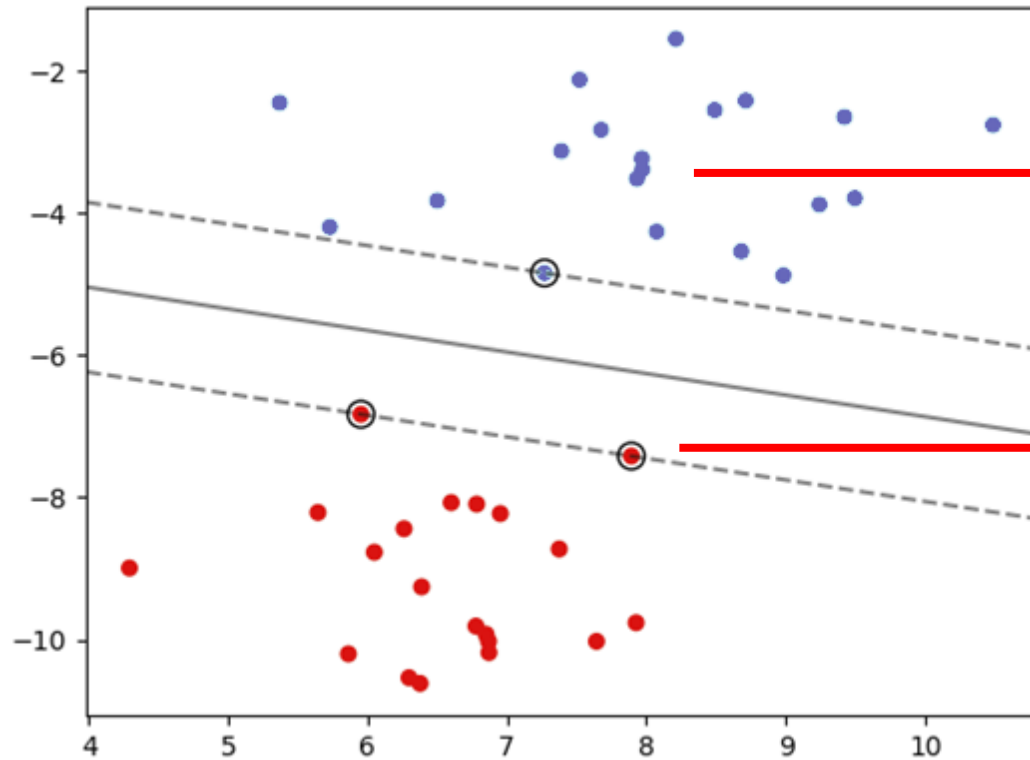
subject to

$$y_n(w^T \phi(x_n) + b) \geq 1 \quad \text{for } n = 1, \dots, N$$

**This is known as the
primal optimization**

- $\phi(x)$ are D-dimensional vectors $\Rightarrow w$ is a D-dimensional *vector*
- Margins determined by nearest data points called *support vectors*

Support Vector Machine: dual problem



All other points are outside the margin and constraints are *loose*:

$$y_n(w^T \phi(x_n) + b) > 1$$

Support vectors are tight to the margin, and satisfy constraints with equality:

$$y_n(w^T \phi(x_n) + b) = 1$$

SVM Dual Problem (high-level idea) Find the support vectors (set of constraints that hold with equality) that induce the max-margin classifier

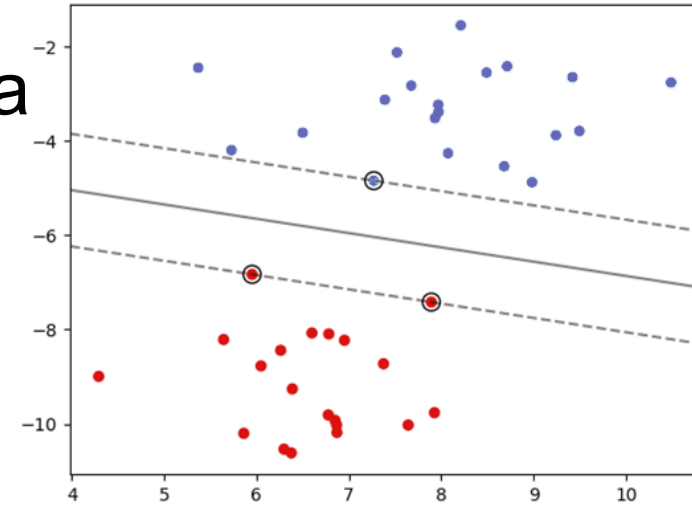
Support Vector Machine: dual variables

Fact The solution of the SVM, w , can be written as a linear combination of features of examples

$$w = \sum_{n=1}^N \lambda_n y_n \phi(x_n)$$



Dual variable for each exemplar n



- Dual variables are nonzero ($\lambda_n > 0$) for any support vector
- Dual variables are zero ($\lambda_n = 0$) for non-support vectors
- Classifier only needs to store support vectors (sparse representation)
- Making a prediction on a new example x , compare

$$y(x) = \sum_{n=1}^N \lambda_n y_n \phi(x)^T \phi(x_n)$$

against zero

Outline

- Basis Functions
- Support Vector Machine Classifier
- **Kernels**

Kernel Functions: saving computation

Recall: nonlinear SVM predicts by

$$y(x) = \sum_{n=1}^N \lambda_n y_n \underbrace{\phi(x)^T \phi(x_n)}_{\text{Interaction with training points in transformed basis space}} \quad (*)$$

Basis transform on new point Basis transform on training point

Drawback $\phi(x)$ may be extremely high dimensional, making $y(x)$ computationally expensive to evaluate using (*)

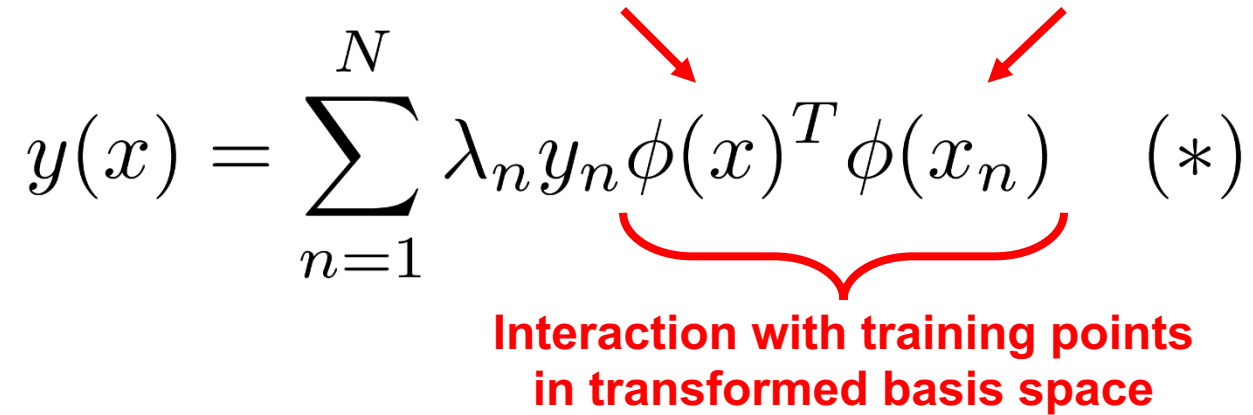
Example

x is 10-dimensional; $\phi(x) = \{\text{all order-10 monomials, e.g. } x_1 \cdot x_2 \cdot \dots \cdot x_{10}\}$
 $\phi(x)$ has $10^{10} = 1$ billion dimensions – too expensive to maintain!

Kernel Functions: saving computation

Recall: nonlinear SVM predicts by

$$y(x) = \sum_{n=1}^N \lambda_n y_n \underbrace{\phi(x)^T \phi(x_n)}_{\text{Interaction with training points in transformed basis space}} \quad (*)$$



Idea Define a new function as the inner product with basis transforms,

**We call this a
“kernel function”**

$$\kappa(x, x_n) = \phi(x)^T \phi(x_n)$$

Intuition of a kernel function: similarity between two inputs

We can now represent the classifier without explicitly maintaining $\phi(x)$'s,

$$y(x) = \sum_{n=1}^N \lambda_n y_n \kappa(x, x_n)$$

Kernel function: definition

- **Definition:** function $K(x, z)$ is said to be a kernel, if there exists some real-valued feature map $\phi(x)$ such that $K(x, z) = \langle \phi(x), \phi(z) \rangle$
Notation: $\langle a, b \rangle$ means dot product
- K is said to be the kernel function associated with feature map ϕ
- Not all functions $K(x, z)$ are kernels!
 - E.g. $K(x, z) = \langle x, z \rangle$ is a kernel function, but $K(x, z) = -\langle x, z \rangle$ is not a kernel function (Why? we will see..)
 - We will see some useful tools for validating / invalidating kernel functions

Basic properties of kernel function

- If K is a kernel function, then:
- Positivity: $K(x, x) \geq 0$ for any x
 - Why? $\langle \phi(x), \phi(x) \rangle \geq 0$ must be true
 - Is $K(x, z) = \max(x, z)$ a kernel function? No, $\max(-1, -1) = -1 < 0$
- Symmetry: $K(x, z) = K(z, x)$ for every x, z
 - Why? $\langle \phi(x), \phi(z) \rangle = \langle \phi(z), \phi(x) \rangle$ must be true
 - Is $K(x, z) = x - z$ a kernel function? No, $K(1, 2) = -1 \neq K(2, 1) = 1$

Kernel function: basic example

- $K(x, z) = 1$ is a valid kernel function
- Why? Find a feature map ϕ such that $\langle \phi(x), \phi(z) \rangle = K(x, z)$
- For this example, we can just define our feature map

$$\phi(x) = (1)$$

so that for any x, z , $\langle \phi(x), \phi(z) \rangle = 1 \times 1 = 1$

Kernel function: an example

- Let $|x| < 1$
- $\phi(x) = (1, x, x^2, x^3, \dots) = (x^n)_{n=0}^{\infty}$
 - An *infinite dimensional* feature map!
 - Impossible to maintain in computer memory explicitly ☹️
- Induced Kernel function:

$$K(x, z) := \langle \phi(x), \phi(z) \rangle = \sum_{n=0}^{\infty} (x \cdot z)^n = \frac{1}{1 - xz}$$

- Takes only constant time to evaluate 😊

Example: Fisher's Iris Dataset

Classify among 3 species of Iris flowers...



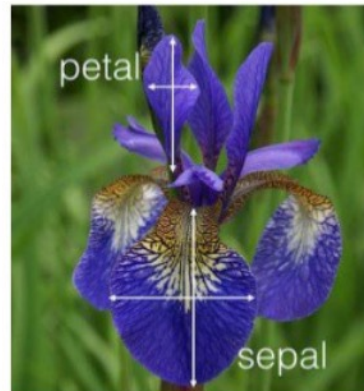
Iris setosa



Iris versicolor



Iris virginica

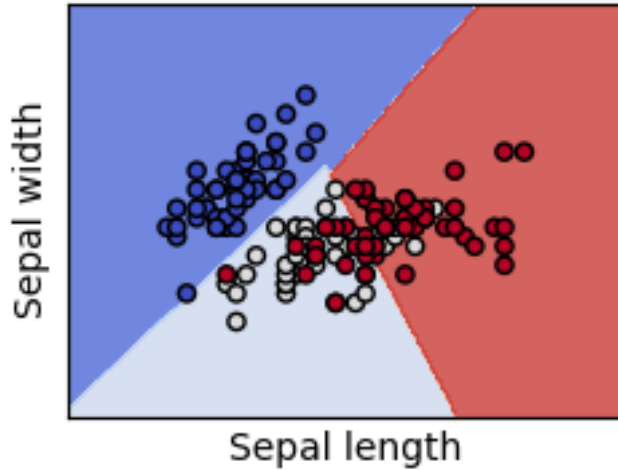


Four features (in centimeters)

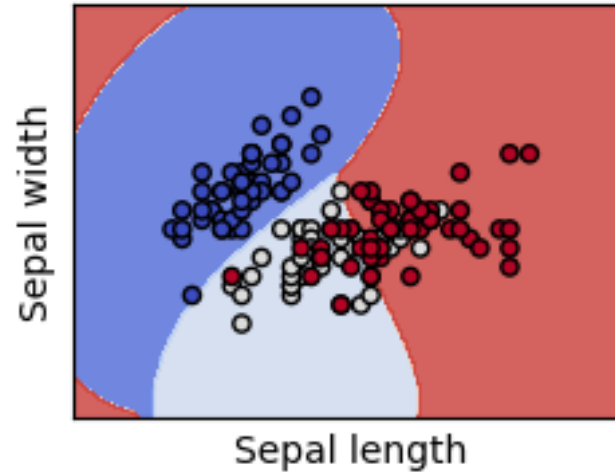
- Petal length / width
- Sepal length / width

Kernel SVM in Scikit Learn

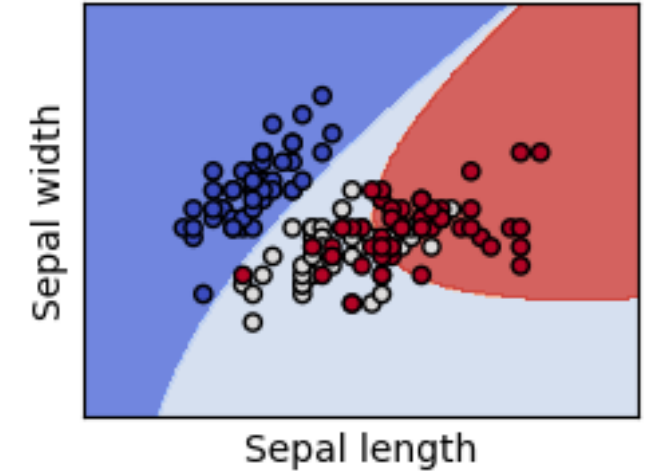
SVC with linear kernel



SVC with RBF kernel



SVC with polynomial (degree 3) kernel



$$\kappa(x, x') = x^T x'$$

$$\kappa(x, x') = \exp(-\gamma \|x - x'\|^2)$$

$$\kappa(x, x') = (x^T x' + c)^3$$

Note: basis function has infinite dimensions

- General kernel-based SVM lives in:

[sklearn.svm.svc\(kernel='kernel_name'\)](#)

- Supports most major kernel types

sklearn.svm.SVC

kernel : {'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'}, default='rbf'

Specifies the kernel type to be used in the algorithm. It must be one of 'linear', 'poly', 'rbf', 'sigmoid', 'precomputed' or a callable. If none is given, 'rbf' will be used. If a callable is given it is used to pre-compute the kernel matrix from data matrices; that matrix should be an array of shape `(n_samples, n_samples)`.

gamma : {'scale', 'auto'} or float, default='scale'

Kernel coefficient for 'rbf', 'poly' and 'sigmoid'.

- if `gamma='scale'` (default) is passed then it uses $1 / (n_features * X.var())$ as value of gamma,
- if 'auto', uses $1 / n_features$.

max_iter : int, default=-1

Hard limit on iterations within solver, or -1 for no limit.

verbose : bool, default=False

Enable verbose output. Note that this setting takes advantage of a per-process runtime setting in libsvm that, if enabled, may not work properly in a multithreaded context.

class_weight : dict or 'balanced', default=None

Set the parameter C of class i to `class_weight[i]*C` for SVC. If not given, all classes are supposed to have weight one. The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as `n_samples / (n_classes * np.bincount(y))`.

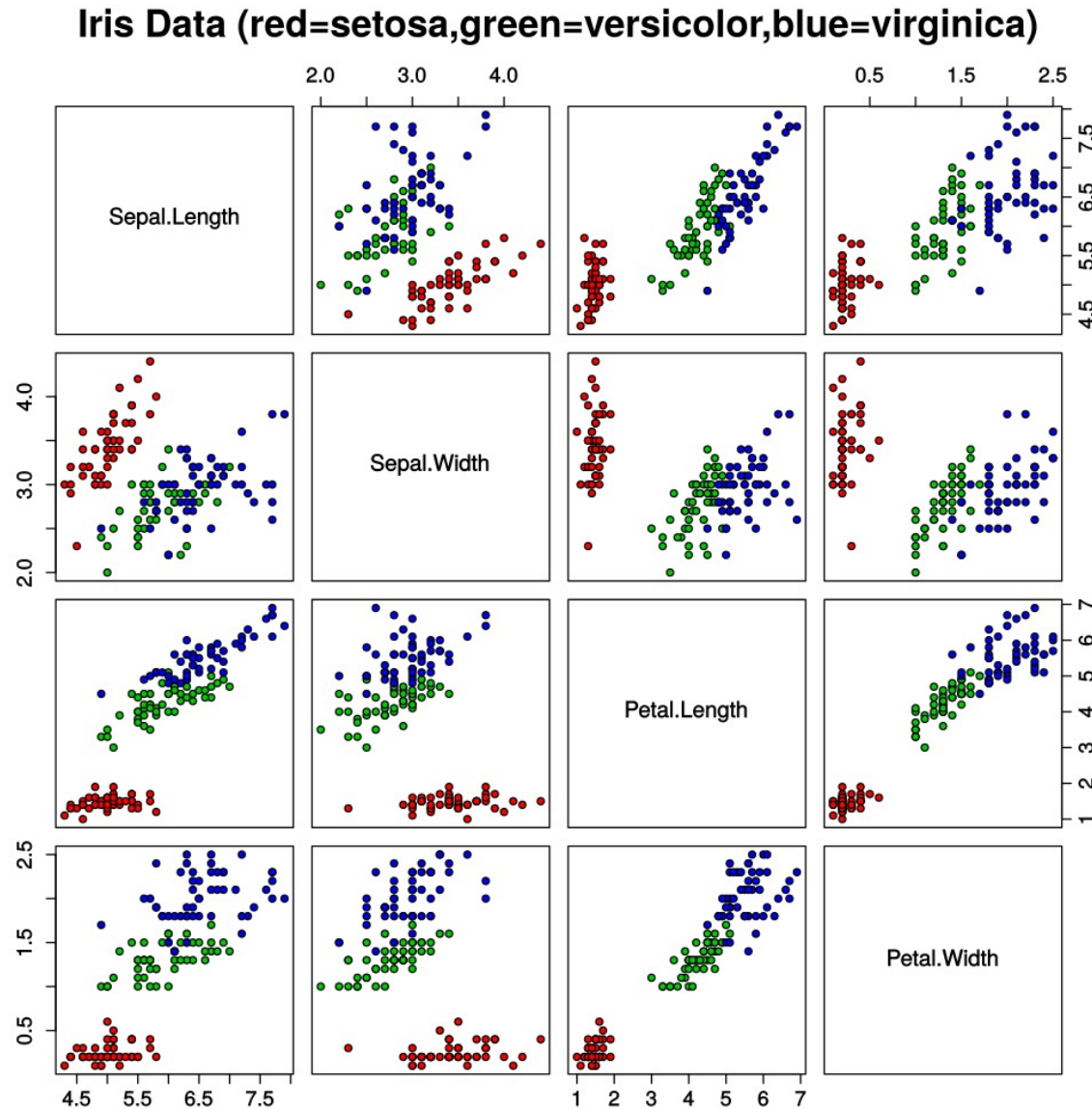
Kernel SVM

- Makes prediction by taking the sign of $y(x) = \sum_{n=1}^N \lambda_n y_n \kappa(x, x_n)$
- Jupyter notebook [here](#)



Dual variable

Example: Fisher's Iris Dataset



*Fairly easy to separate **setosa** from others using a linear classifier*

Need to use nonlinear basis / kernel to better separate other classes

Example: Fisher's Iris Dataset

Train 8-degree polynomial kernel SVM classifier,

```
from sklearn.svm import SVC
svclassifier = SVC(kernel='poly', degree=8)
svclassifier.fit(X_train, y_train)
```

Generate predictions on held-out test data,

```
y_pred = svclassifier.predict(X_test)
```

Show confusion matrix and classification accuracy,

```
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
[[11  0  0]
 [ 0 12  1]
 [ 0  0  6]]
```



True class = versicolor, predicted class = virginica

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	11
Iris-versicolor	1.00	0.92	0.96	13
Iris-virginica	0.86	1.00	0.92	6
avg / total	0.97	0.97	0.97	30

Linear kernel

Example The *linear basis* $\phi(x) = x$ produces the kernel,

$$\kappa(x, x') = \phi(x)^T \phi(x') = x^T x'$$

This is called the linear kernel

Polynomial kernels

- $\kappa(x, z) = (1 + \langle x, z \rangle)^k$ Q: is this a valid kernel?
- E.g., if $d = 2$, $x = (x_1, x_2)$ and $z = (z_1, z_2)$, $k = 2$,

$$\begin{aligned} & (1 + x_1 z_1 + x_2 z_2)^2 \\ &= 1 + x_1^2 z_1^2 + x_2^2 z_2^2 + 2x_1 z_1 + 2x_2 z_2 + 2x_1 x_2 z_2 z_1 \\ &= \langle \phi(x), \phi(z) \rangle \end{aligned}$$

where $\phi(u) = (u_1^2, u_2^2, \sqrt{2}u_1, \sqrt{2}u_2, \sqrt{2}u_1 u_2, 1)$

- **Exercise** can we find feature maps for κ for general k , d ?
- **Exercise** can we find other ϕ associated with κ ?

Yes

Many: E.g., $\phi(u) = (u_1^2, u_2^2, u_1, u_1, \sqrt{2}u_2, \sqrt{2}u_1 u_2, 1)$

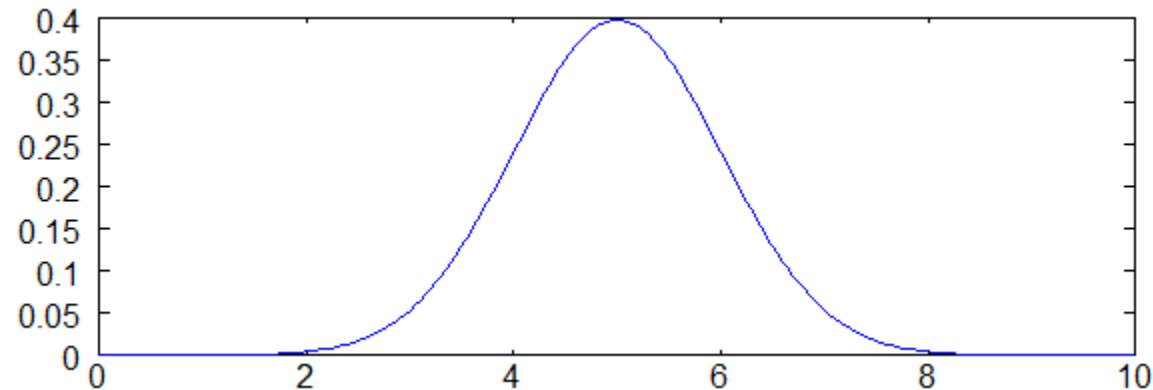
Gaussian kernel

Example Gaussian kernel models similarity according to an unnormalized Gaussian distribution,

$$K(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right)$$

Note Despite the name, this is **not** a Gaussian probability density.

Also called a *radial basis function* (RBF)



Validity of Gaussian/RBF kernels (580 only)

- $K(x, z) = \exp\left(-\frac{\|x-z\|^2}{2\sigma^2}\right)$ (often parameterized as $\exp(-\gamma\|x-z\|^2)$)
- How can we show that this is a valid kernel?

=> We should find $\phi(x)$ that results in
 $K(x, x') = \langle \phi(x), \phi(x') \rangle$

Assume $x \in R^1$ and $\gamma > 0$.

$$\begin{aligned} e^{-\gamma\|x_i-x_j\|^2} &= e^{-\gamma(x_i-x_j)^2} = e^{-\gamma x_i^2 + 2\gamma x_i x_j - \gamma x_j^2} \\ &= e^{-\gamma x_i^2 - \gamma x_j^2} \left(1 + \frac{2\gamma x_i x_j}{1!} + \frac{(2\gamma x_i x_j)^2}{2!} + \frac{(2\gamma x_i x_j)^3}{3!} + \dots\right) \\ &= e^{-\gamma x_i^2 - \gamma x_j^2} \left(1 \cdot 1 + \sqrt{\frac{2\gamma}{1!}} x_i \cdot \sqrt{\frac{2\gamma}{1!}} x_j + \sqrt{\frac{(2\gamma)^2}{2!}} x_i^2 \cdot \sqrt{\frac{(2\gamma)^2}{2!}} x_j^2 \right. \\ &\quad \left. + \sqrt{\frac{(2\gamma)^3}{3!}} x_i^3 \cdot \sqrt{\frac{(2\gamma)^3}{3!}} x_j^3 + \dots\right) = \phi(x_i)^T \phi(x_j), \end{aligned}$$

where

$$\phi(x) = e^{-\gamma x^2} \left[1, \sqrt{\frac{2\gamma}{1!}} x, \sqrt{\frac{(2\gamma)^2}{2!}} x^2, \sqrt{\frac{(2\gamma)^3}{3!}} x^3, \dots\right]^T.$$

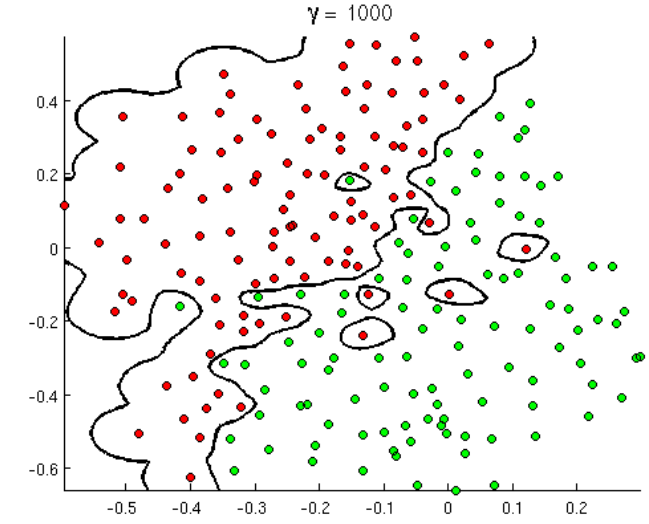
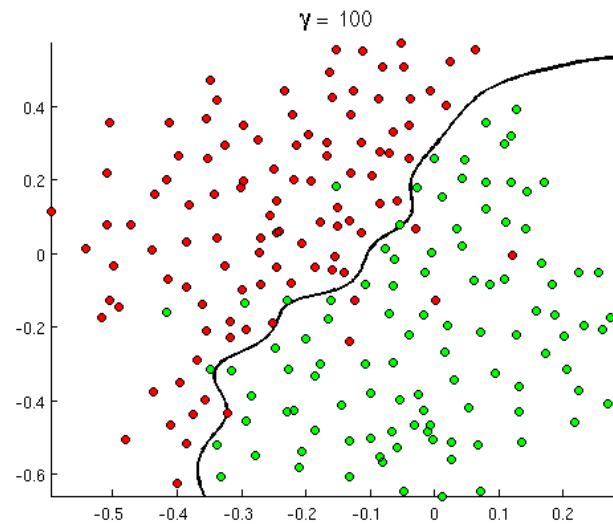
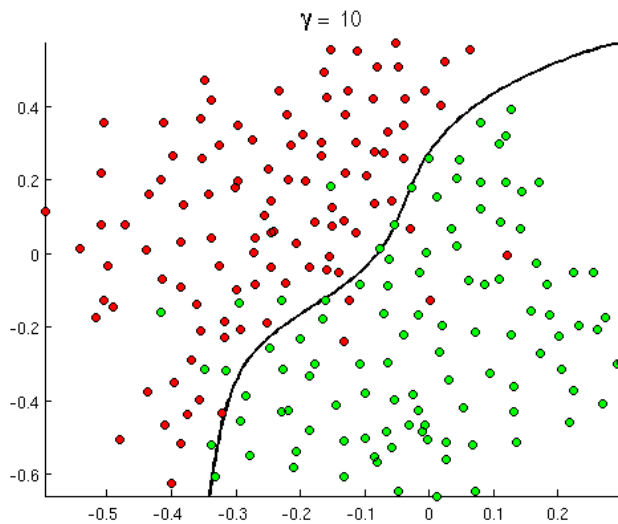
Gaussian kernel SVM is similar to weighted k-NN

- $\gamma = \frac{1}{2\sigma^2}$

recall how kernel SVM make predictions:
 $w^\top \phi(x_*) = \sum_i \alpha_i y_i K(x_i, x_*)$

Similar to weighted k-NN

- $\arg \max_y \sum_{i \in N(x_*)} w_i 1\{y_i = y\}$
- e.g., $w_i = \exp\left(-\gamma \cdot (d(x_i, x_*))^2\right)$



- Larger $\gamma \Rightarrow$ smaller $\sigma^2 \Rightarrow$ more likely to overfit
- A popular heuristic: choose $\sigma = \text{median}(\|x_i - x_j\|, i \neq j)$

Another way to recognize a kernel

Fact (Mercer's Theorem): κ is a valid kernel function if and only if κ satisfies Mercer's condition, i.e., Given *any* set of data $S = \{x_i\}_{i=1}^n$, its $n \times n$ **Gram matrix**,

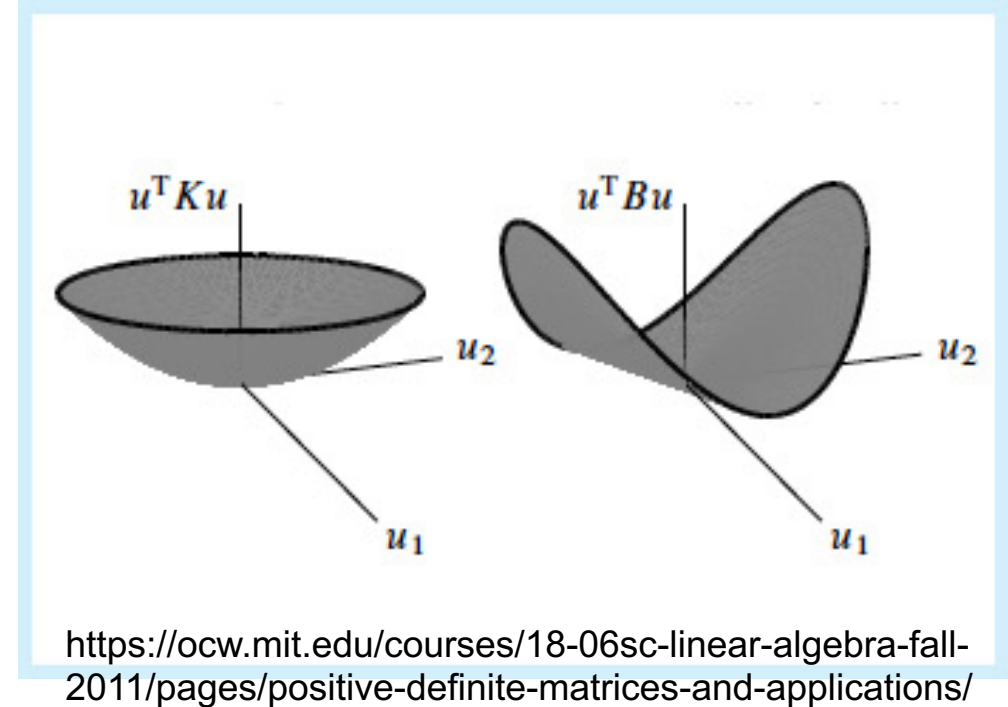
$$\mathbf{K}_S = \begin{pmatrix} \kappa(x_1, x_1) & \kappa(x_1, x_2) & \dots & \kappa(x_1, x_n) \\ \kappa(x_2, x_1) & \kappa(x_2, x_2) & \dots & \kappa(x_2, x_n) \\ \vdots & \vdots & \vdots & \vdots \\ \kappa(x_n, x_1) & \kappa(x_n, x_2) & \dots & \kappa(x_n, x_n) \end{pmatrix}$$

Is a *symmetric positive semidefinite (PSD) matrix*.

Matrix M is symmetric if $M_{i,j} = M_{j,i}$ for any i, j

Math interlude: symmetric positive semi-definite matrices

- A square symmetric matrix $M \in \mathbb{R}^{d \times d}$ is said to be positive semi-definite (PSD), if for any vector $v \in \mathbb{R}^d$, $v^\top M v \geq 0$
- $d = 1$, PSD matrices are just nonnegative numbers
- They yield quadratic functions that “curve up” in all directions
- M is PSD $\Leftrightarrow$ all eigenvalues of M are nonnegative



Example

- Suppose inputs x are real numbers
- Is $\kappa(x, z) = \max(x, z)$ a valid kernel function?

No -- It does not satisfy positivity

let's see how to use Mercer's condition to check it in another way

- The dataset $S = \{-1\}$ has 1x1 Gram matrix

$$\mathbf{K}_S = (\max(-1, -1)) = (-1)$$

- It is not PSD, since we can take $v = (1)$ and
$$v^\top \mathbf{K}_S v = 1 \times (-1) \times 1 = -1 < 0$$
- Thus, κ fails Mercer's condition

Another way to recognize a kernel

Fact (Mercer's Theorem): κ is a valid kernel function if and only if κ satisfies Mercer's condition, i.e., Given *any* set of data $S = \{x_i\}_{i=1}^n$, its $n \times n$ **Gram matrix**,

$$\mathbf{K}_S = \begin{pmatrix} \kappa(x_1, x_1) & \kappa(x_1, x_2) & \dots & \kappa(x_1, x_n) \\ \kappa(x_2, x_1) & \kappa(x_2, x_2) & \dots & \kappa(x_2, x_n) \\ \vdots & \vdots & \vdots & \vdots \\ \kappa(x_n, x_1) & \kappa(x_n, x_2) & \dots & \kappa(x_n, x_n) \end{pmatrix}$$

Is a *symmetric positive semidefinite (PSD) matrix*.

This is actually more useful for showing a function to NOT be a kernel...

Example

- Suppose inputs $x, y \geq 0$
- Is $\kappa(x, y) = \max(x, y)$ a valid kernel function?
- After trying a few ϕ 's to see if $\kappa(x, y) = \langle \phi(x), \phi(y) \rangle$ and failing, may want to think about showing the opposite
- Guess $S = \{0, 2\} \Rightarrow \mathbf{K}_S = \begin{pmatrix} \kappa(0,0) & \kappa(0,2) \\ \kappa(2,0) & \kappa(2,2) \end{pmatrix} = \begin{pmatrix} 0 & 2 \\ 2 & 2 \end{pmatrix}$
 - $\mathbf{K}_S$ is not PSD. Why?
 - Method 1: find v such that $v^\top \mathbf{K}_S v < 0$
 - Method 2: check that some eigenvalue of $\mathbf{K}_S$ is < 0

Example

$$\mathbf{K}_S = \begin{pmatrix} \kappa(0,0) & \kappa(0,2) \\ \kappa(2,0) & \kappa(2,2) \end{pmatrix} = \begin{pmatrix} 0 & 2 \\ 2 & 2 \end{pmatrix}$$

- $\mathbf{K}_S$ is not PSD. Why?
- Method 1: find v such that $v^\top \mathbf{K}_S v < 0$
- Requires some trial and error...
- E.g. by trying $v = (-1, 1)$,

$$v^\top \mathbf{K}_S v = (-1, 1) \begin{pmatrix} 0 & 2 \\ 2 & 2 \end{pmatrix} \begin{pmatrix} -1 \\ 1 \end{pmatrix} = -2$$

Example

$$\mathbf{K}_S = \begin{pmatrix} \kappa(0,0) & \kappa(0,2) \\ \kappa(2,0) & \kappa(2,2) \end{pmatrix} = \begin{pmatrix} 0 & 2 \\ 2 & 2 \end{pmatrix}$$

- $\mathbf{K}_S$ is not PSD. Why?
 - Method 2: check that some eigenvalue of $\mathbf{K}_S$ is < 0
 - Avoids trial and error
 - Use `np.linalg.eigvals`
 - See notebook [here](#)

Techniques for Constructing New Kernels.

Given valid kernels $k_1(\mathbf{x}, \mathbf{x}')$ and $k_2(\mathbf{x}, \mathbf{x}')$, the following new kernels will also be valid:

$$k(\mathbf{x}, \mathbf{x}') = ck_1(\mathbf{x}, \mathbf{x}') \quad (6.13)$$

$$k(\mathbf{x}, \mathbf{x}') = f(\mathbf{x})k_1(\mathbf{x}, \mathbf{x}')f(\mathbf{x}') \quad (6.14)$$

$$k(\mathbf{x}, \mathbf{x}') = q(k_1(\mathbf{x}, \mathbf{x}')) \quad (6.15)$$

$$k(\mathbf{x}, \mathbf{x}') = \exp(k_1(\mathbf{x}, \mathbf{x}')) \quad (6.16)$$

$$k(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}') + k_2(\mathbf{x}, \mathbf{x}') \quad (6.17)$$

$$k(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}')k_2(\mathbf{x}, \mathbf{x}') \quad (6.18)$$

$$k(\mathbf{x}, \mathbf{x}') = k_3(\phi(\mathbf{x}), \phi(\mathbf{x}')) \quad (6.19)$$

$$k(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{A} \mathbf{x}' \quad (6.20)$$

$$k(\mathbf{x}, \mathbf{x}') = k_a(\mathbf{x}_a, \mathbf{x}'_a) + k_b(\mathbf{x}_b, \mathbf{x}'_b) \quad (6.21)$$

$$k(\mathbf{x}, \mathbf{x}') = k_a(\mathbf{x}_a, \mathbf{x}'_a)k_b(\mathbf{x}_b, \mathbf{x}'_b) \quad (6.22)$$

Why Kernel Functions?

At this point you might be confused...

- We learned how to fit linear models
- We learned how to introduce nonlinearities by using basis functions
- Kernels are just inner products of basis functions

...then why do we need kernels?

First answer: using kernels can save a huge amount of computation

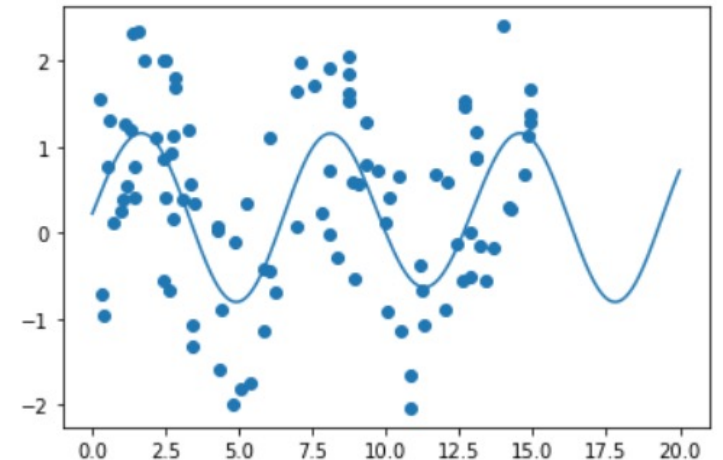
Any other considerations?

Why Kernel Functions?

- Can directly specify kernel function without knowing basis functions
- Kernels can be more intuitive to specify since they capture meaningful distance / difference between two data points
- **Example 1** using kernel to capture periodic similarities

$$k(x_i, x_j) = \exp \left(-\frac{2 \sin^2(\pi d(x_i, x_j)/p)}{l^2} \right)$$

Largest when $d(x_i, x_j)$ is a multiple of p



Example usage: predicting website traffic based on time

Why Kernel Functions?

- Can directly specify kernel function without knowing basis functions
- Kernels can be more intuitive to specify since they capture meaningful distance / difference between two data points

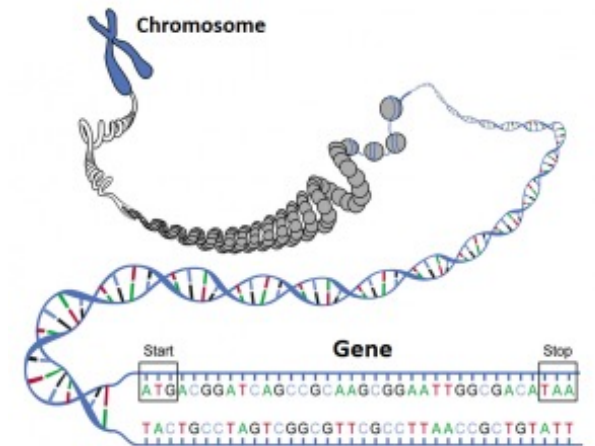
- **Example 2** String kernels

- $k_n(x, y) = \# \text{common length} - n \text{ substrings of } x \text{ and } y$

- $x = \text{'ATCG'}$

- $y = \text{'ATTC'}$

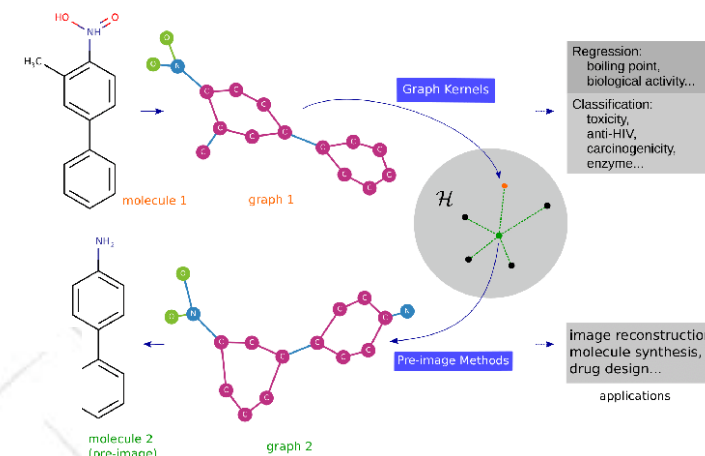
- $k_2(x, y) = 2$, since they have 2 common length-2 substrings 'AT' and 'TC'



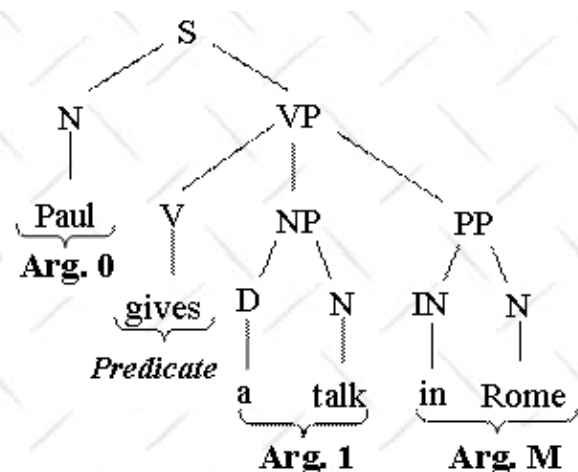
Why Kernel Functions?

- Can directly specify kernel function without knowing basis functions
- Kernels can be more intuitive to specify since they capture meaningful distance / difference between two data points

- **Example 3** Graph kernels



- **Example 4** Tree kernels



Kernel Ridge Regression

Recall the solution of L2-regularized linear regression (ridge regression),

$$\mathbf{\Phi} = \begin{pmatrix} \phi_1(x_1) & \dots & \phi_M(x_1) \\ \phi_1(x_2) & \dots & \phi_M(x_2) \\ \vdots & \vdots & \vdots \\ \phi_1(x_N) & \dots & \phi_M(x_N) \end{pmatrix} = \begin{pmatrix} - & \phi(x_1) & - \\ - & \phi(x_2) & - \\ \vdots & \vdots & \vdots \\ - & \phi(x_N) & - \end{pmatrix} \quad w^{\text{ridge}} = (\mathbf{\Phi}^T \mathbf{\Phi} + \lambda I)^{-1} \mathbf{\Phi}^T \mathbf{y}$$

Define the Gram matrix and vector as,

$$\mathbf{K} = \mathbf{\Phi} \mathbf{\Phi}^T = \begin{pmatrix} \kappa(x_1, x_1) & \kappa(x_1, x_2) & \dots & \kappa(x_1, x_n) \\ \kappa(x_2, x_1) & \kappa(x_2, x_2) & \dots & \kappa(x_2, x_n) \\ \vdots & \vdots & \vdots & \vdots \\ \kappa(x_n, x_1) & \kappa(x_n, x_2) & \dots & \kappa(x_n, x_n) \end{pmatrix}$$

$$\mathbf{k}(\mathbf{x})^T = (\phi(x)^T \phi(x_1), \dots, \phi(x)^T \phi(x_n))$$

Kernel Ridge Regression

The learned regression function (for a new point) is then,

$$y(x) = \phi(x)^\top w$$

Solution to ridge regression $= \phi(x)^T (\Phi^T \Phi + \lambda I)^{-1} \Phi^T \mathbf{y}$

Nontrivial linear algebra identity $= \phi(x)^T \Phi^T (\Phi \Phi^T + \lambda I)^{-1} \mathbf{y}$

Substitute kernel $= \mathbf{k}(\mathbf{x})^T (\mathbf{K} + \lambda I)^{-1} \mathbf{y}$ **Also known as the dual formulation of linear regression**

Can now do regression without explicitly specifying basis functions

Kernel Ridge Regression

Kernel representation requires inversion of NxN matrix

Primal

$$\Phi = \begin{pmatrix} \phi_1(x_1) & \dots & \phi_M(x_1) \\ \phi_1(x_2) & \dots & \phi_M(x_2) \\ \vdots & \vdots & \vdots \\ \phi_1(x_N) & \dots & \phi_M(x_N) \end{pmatrix}$$

$$w = (\underbrace{\Phi^T \Phi + \lambda I}_{\text{MxM Matrix Inversion}})^{-1} \Phi^T y$$

MxM Matrix Inversion
O(M³)

Dual

$$\mathbf{K} = \begin{pmatrix} \kappa(x_1, x_1) & \kappa(x_1, x_2) & \dots & \kappa(x_1, x_n) \\ \kappa(x_2, x_1) & \kappa(x_2, x_2) & \dots & \kappa(x_2, x_n) \\ \vdots & \vdots & \vdots & \vdots \\ \kappa(x_n, x_1) & \kappa(x_n, x_2) & \dots & \kappa(x_n, x_n) \end{pmatrix}$$

$$y(x) = \mathbf{k}(\mathbf{x})^T (\underbrace{\mathbf{K} + \lambda I}_{\text{NxN Matrix Inversion}})^{-1} \mathbf{y}$$

NxN Matrix Inversion
O(N³)

#training data N vs. #basis functions M

sklearn.kernel_ridge.KernelRidge

alpha : float or array-like of shape (n_targets,), default=1.0

Regularization strength; must be a positive float. Regularization improves the conditioning of the problem and reduces the variance of the estimates. Larger values specify stronger regularization. Alpha corresponds to $1 / (2C)$ in other linear models such as `LogisticRegression` or `LinearSVC`. If an array is passed, penalties are assumed to be specific to the targets. Hence they must correspond in number. See [Ridge regression and classification](#) for formula.

kernel : str or callable, default="linear"

Kernel mapping used internally. This parameter is directly passed to `pairwise_kernel`. If `kernel` is a string, it must be one of the metrics in `pairwise.PAIRWISE_KERNEL_FUNCTIONS`. If `kernel` is "precomputed", X is assumed to be a kernel matrix. Alternatively, if `kernel` is a callable function, it is called on each pair of instances (rows) and the resulting value recorded. The callable should take two rows from X as input and return the corresponding kernel value as a single number. This means that callables from `sklearn.metrics.pairwise` are not allowed, as they operate on matrices, not single samples. Use the string identifying the kernel instead.

gamma : float, default=None

Gamma parameter for the RBF, laplacian, polynomial, exponential chi2 and sigmoid kernels. Interpretation of the default value is left to the kernel; see the documentation for `sklearn.metrics.pairwise`. Ignored by other kernels.

Example: Kernel Ridge Regression

Generate some sinusoidal (periodic) data,

```
X = 15 * rng.rand(100, 1)
y = np.sin(X).ravel()
y += 3 * (0.5 - rng.rand(X.shape[0])) # add noise
```

Define an exponentiated sinusoidal kernel (what kind of similarity does this capture?)

```
from sklearn.gaussian_process.kernels import ExpSineSquared
kernel = ExpSineSquared(length_scale=4.64, periodicity=12.9)
```

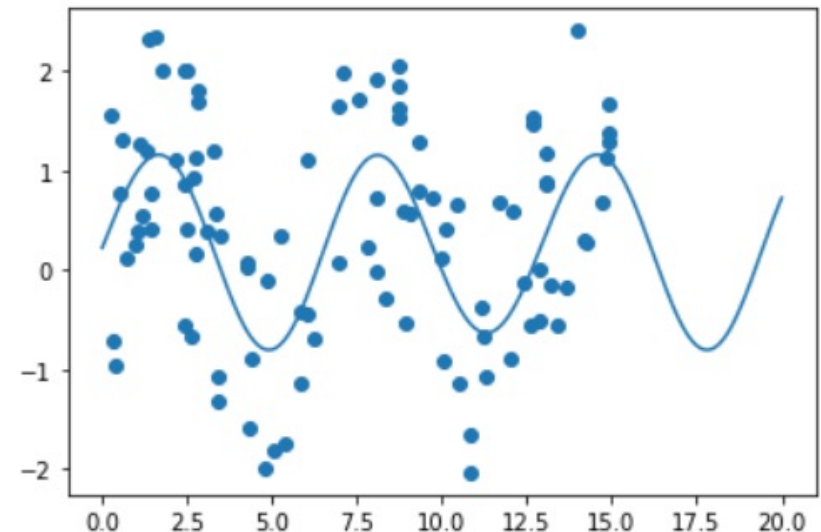
$$\exp\left(-\frac{2\sin^2(\pi d(x_i, x_j)/p)}{l^2}\right)$$

Fit kernel ridge regression,

```
from sklearn.kernel_ridge import KernelRidge
kr = KernelRidge(kernel=kernel, alpha=0.001).fit(X, y)
```

Plot results,

```
X_plot = np.linspace(0, 20, 10000)[: , None]
y_kr = kr.predict(X_plot)
plt.scatter(X, y)
plt.plot(X_plot, y_kr)
plt.show()
```



Kernelized Perceptron algorithm

- How to combine the Perceptron algorithm with a nonlinear feature mapping $\phi: \mathcal{X} \rightarrow \mathbb{R}^D$?
- Recall the Perceptron algorithm, run with $\phi(x)$'s:

Algorithm 29 PERCEPTONTRAIN($\mathbf{D}$, $MaxIter$)

```
1:  $w \leftarrow 0, b \leftarrow 0$  // initialize weights and bias
2: for  $iter = 1 \dots MaxIter$  do
3:   for all  $(x, y) \in \mathbf{D}$  do
4:      $a \leftarrow w \cdot \phi(x) + b$  // compute activation for this example
5:     if  $ya \leq 0$  then
6:        $w \leftarrow w + y \phi(x)$  // update weights
7:        $b \leftarrow b + y$  // update bias
8:     end if
9:   end for
10: end for
11: return  $w, b$ 
```

- Is it possible to implement this without ever explicitly computing ϕ ?
- Suppose ϕ induces a kernel K

Kernelized Perceptron algorithm

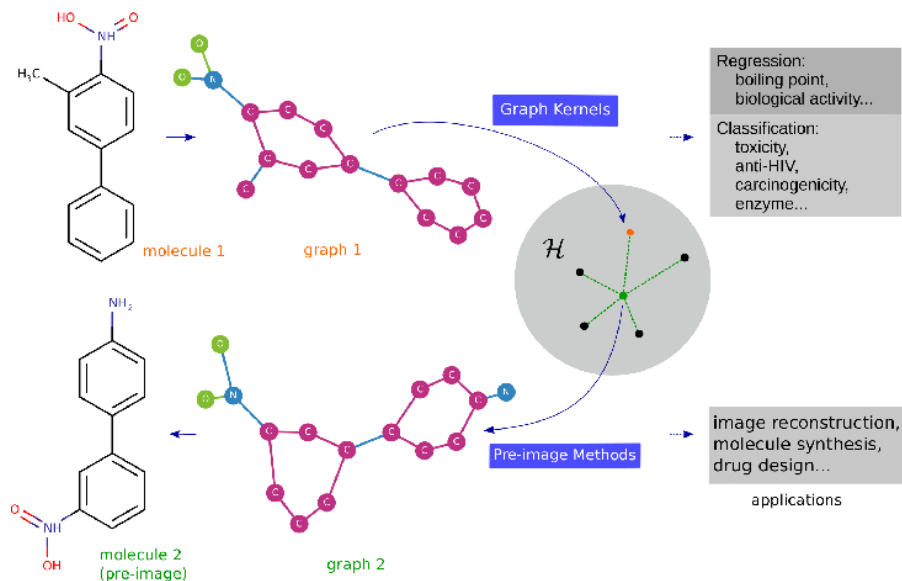
- Key observation: throughout the run of the Perceptron algorithm, w always has the form $\alpha_1 \phi(x_1) + \dots + \alpha_n \phi(x_n)$
- Key algorithmic idea: instead of *explicitly* maintaining $w \in \mathbb{R}^D$, we *implicitly* maintain it by maintaining its linear combination coefficient $(\alpha_1, \dots, \alpha_n) \in \mathbb{R}^n$!

Algorithm 30 KERNELIZEDPERCEPTRONTRAIN($\mathbf{D}$, $MaxIter$)

```
1:  $\alpha \leftarrow 0, b \leftarrow 0$  // initialize coefficients and bias
2: for  $iter = 1 \dots MaxIter$  do
3:   for all  $(x_n, y_n) \in \mathbf{D}$  do
4:      $a \leftarrow \sum_m \alpha_m \phi(x_m) \cdot \phi(x_n) + b$  // compute activation for this example
5:     if  $y_n a \leq 0$  then calculate this by directly evaluating  $K(x_m, x_n)$ 
6:        $\alpha_n \leftarrow \alpha_n + y_n$  // update coefficients
7:        $b \leftarrow b + y$  // update bias
8:     end if
9:   end for
10: end for
11: return  $\alpha, b$ 
```

Kernel methods: summary

- Kernels may be more intuitive to specify than basis functions
- Many standard algorithms have their kernelized versions
- Computational complexity:
 - Avoids dependence on basis function dimension 😊
 - oftentimes $O(n^2)$, $n = \text{\#training examples}$ 😞



Kernel methods: summary

- What if we have a good choice of kernel κ but cannot stand $O(n^2)$ time cost?
 - Find basis function ϕ such that $\kappa(x, x') = \langle \phi(x), \phi(x') \rangle$, or even just approximately equal
 - (NeurIPS'17 test of time paper)
- Modern perspective:
 - Kernel methods are useful for understanding training neural networks (later in the course)

Random Features for Large-Scale Kernel Machines

Ali Rahimi and Ben Recht

GRADIENT DESCENT PROVABLY OPTIMIZES
OVER-PARAMETERIZED NEURAL NETWORKS

Simon S. Du*
Machine Learning Department
Carnegie Mellon University
ssdu@cs.cmu.edu

Xiyu Zhai*
Department of EECS
Massachusetts Institute of Technology
xiyuzhai@mit.edu

Barnabás Póczos
Machine Learning Department
Carnegie Mellon University
bapozos@cs.cmu.edu

Aarti Singh
Machine Learning Department
Carnegie Mellon University
aartisinhg@cmu.edu

**Neural Tangent Kernel:
Convergence and Generalization in Neural Networks**

Arthur Jacot
École Polytechnique Fédérale de Lausanne
arthur.jacot@netopera.net

Franck Gabriel
Imperial College London and École Polytechnique Fédérale de Lausanne
franckrgabriel@gmail.com

Clément Hongler
École Polytechnique Fédérale de Lausanne
clement.hongler@gmail.com